

מאפס למומחה: ML Engineering ו־MLOps

מסלול למידה מלא לאדם בלי רקע בתחום, עד רמה שמאפשרת לעבוד כמהנדס למידת מכונה או כמהנדס MLOps. כל פרק מסביר בעיה, מראה קוד שרץ, ומסתיים בתרגיל שאפשר לבדוק את עצמך מולו.

≈ 600 שעות לימוד

3 פרויקטים לתיק עבודות

קוד שנבדק והורץ

4 נספחים

40 פרקים

ללא ידע מוקדם

מטרת הפרק: להבין מה המדריך נותן, מה הוא לא נותן, ואיך לעבוד איתו כך שבסוף תהיה לך יכולת ולא רק ידע.

המטרה של המדריך היא שתדע לבנות מודל, לבדוק אותו, להפוך אותו לשירות, להפעיל אותו בייצור, ולתקן אותו כשמשו משתבש. הקריאה תיתן לך את ההבנה. המומחיות והמוכנות לעבודה יגיעו מהשלמת התרגילים והפרויקטים, לא מהמעבר על הטקסט.

המדריך בנוי בשישה חלקים שמצטברים זה על זה. אל תדלג קדימה כי נושא נשמע מעניין יותר: הפרקים המתקדמים מניחים שאתה כבר יודע לקרוא שגיאה, לכתוב פונקציה, ולהסביר מה קורה בתוך Pipeline.

שלושת הכללים שיקבעו אם זה יעבוד

1. **קוד לפני קריאה.** כל דוגמה במדריך מיועדת להרצה אצלך במחשב. קוד שקראת ולא הרצת לא נשמר.
2. **שינוי אחד בכל פעם.** אחרי שדוגמה עובדת, שנה בה דבר אחד ונסה לנחש מראש מה ישתנה. הפער בין הניחוש לתוצאה הוא הלמידה.
3. **הסבר בקול.** אם אינך יכול להסביר בשתי דקות מה הקוד עושה ולמה, עדיין לא סיימת את היחידה.

מה השתנה במהדורה הזאת

תוספת	למה היא נחוצה
פרק ניתוח טעויות (17)	ההבדל הגדול בין מתחיל למקצוען הוא לא בחירת האלגוריתם, אלא היכולת לקרוא את הטעויות של המודל ולהחליט מה לעשות.
פרק LLMOps (34)	מערכות מבוססות מודלי שפה נכנסות לייצור מהר, והתפעול שלהן שונה מהותית ממודל טבלאי.
פרק אבטחה, פרטיות וממשל (31)	שאלות רגולציה, הרשאות ותיעוד החלטות מגיעות בכל ראיון לתפקיד בכיר, ובכל ארגון גדול.
מעבדה שרצה באמת	קוד הפרויקט יושב בתיקייה <code>labs/iris-service</code> במאגר, עם <code>Makefile</code> , בדיקות ו- <code>Dockerfile</code> . הוא הורץ ונבדק, לא רק נכתב.
נספחים	מילון מונחים, צ'קליסט שחרור לייצור, רשימת מקורות ו-20 הטעויות הנפוצות במקום אחד.
צ'קליסטים מסומנים	הסימון נשמר בדפדפן שלך, כך שאפשר לעקוב אחר ההתקדמות לאורך חודשים.
גרסת שמע	שמונה פרקי פודקאסט, כ-41 דקות, להאזנה בהליכה או בנסיעה. ההקראה נעשית במנוע הדיבור של המכשיר, בלי התקנה.

חלק א' · פרקים 1-9

הבסיס ההנדסי

מה המקצוע, איך עובד מחשב, סביבת עבודה, Python, קוד תחזוקתי, Linux, Git, רשתות, SQL, pandas ומתמטיקה.

חלק ב' · פרקים 10-17

למידת מכונה

מה מודל לומד, הגדרת בעיה, נתונים ודליפה, חלוקות, אלגוריתמים, מדדים, תכונות וניתוח טעויות.

חלק ג' · פרקים 18-25

מ־Notebook לשירות

אימון, API, בדיקות, ניסויים, שחזוריות, Docker, CI/CD ואסטרטגיות פריסה.

חלק ד' · פרקים 26-31

נתונים, תשתית ותפעול

הנדסת נתונים, אחסון ו־Feature Store, Kubernetes, ניטור, טיפול בתקלות ואבטחה.

חלק ה' · פרקים 32-36

מתקדם

למידה עמוקה, NLP ו־RAG, LLMOps, ביצועים ועלויות, וענן.

חלק ו' · פרקים 37-40

מהמדריך לעבודה

תיק עבודות, תוכנית עבודה שבועית, שאלות ראיון ומבחן מוכנות.

מה המדריך לא עושה

- הוא לא מחליף תרגול. אין קיצור דרך בין קריאה לבין יכולת.
- הוא לא מבטיח קבלה לעבודה. הוא מכין אותך לשיחה מקצועית ולמשימה מעשית.
- הוא לא מקבע כלי מסוים. הכלים משתנים; הבעיות שהם פותרים נשארות.
- המספרים בדוגמאות הם לימודיים. אל תצטט אותם כביצועים של מערכת אמיתית.

אם יש לך רקע חלקי

יודע לתכנת אבל לא מכיר ML? התחל בפרק 4 לרענן, ועבור לחלק ב'. עובד בתשתיות או DevOps? קרא את חלק ב' בעיון והתמקד אחר כך בחלקים ג' וד', שם הידע שלך שווה הכי הרבה.

חלק א' · פרקים 1-9

הבסיס ההנדסי

לפני שמאמנים מודל צריך לדעת איפה הקוד רץ, איפה הנתונים יושבים, ואיך מוציאים מהם תשובה. החלק הזה בונה את הקרקע שכל השאר עומד עליה.

01 לאיזה מקצוע אתה נכנס

מטרת הפרק: לדעת מה ההבדל בין התפקידים בתחום, ולבחור כיוון התמחות מודע במקום ללמוד הכול חצי.

מערכת למידת מכונה היא מערכת תוכנה שמשתמשת בדפוסים שנלמדו מנתונים כדי להפיק תחזיות או החלטות.

לדוגמה, חברה רוצה להעריך כמה זמן יידרש להקים כנס. יש לה נתונים מאירועים קודמים: מספר משתתפים, מספר חדרים, סוג הציוד, גודל הצוות וזמן ההקמה בפועל. מודל יכול ללמוד קשרים בין הנתונים האלה ולספק הערכה לאירוע חדש. הדוגמה הזאת תלווה אותנו לאורך כל המדריך.

אבל כדי שהיכולת הזאת תהיה שימושית, מישוהו צריך לאסוף את הנתונים, לבדוק אותם, לאמן את המודל, לחבר אותו לאפליקציה, ולעקוב אחר הטעויות שלו לאורך זמן. שם נמצאת העבודה שלך.

תפקיד	השאלה המרכזית	עבודה אופיינית
Data Analyst	מה קרה ומדוע?	SQL, דוחות, מדדים וניתוח עסקי
Data Scientist	האם אפשר ללמוד דפוס שימושי מהנתונים?	ניסויים, סטטיסטיקה ומודלים
ML Engineer	איך בונים יכולת ML שעובדת במוצר?	נתונים, אימון, הערכה, קוד ושירות חיזוי
MLOps Engineer	איך מפתחים ומפעילים יכולות ML באופן אמין?	אוטומציה, גרסאות, פריסה, ניטור ותשתיות
ML Platform Engineer	איך מאפשרים לצוותים רבים לעבוד ביעילות?	פלטפורמות משותפות, הרשאות, משאבים וכלי פיתוח
Research Engineer	איך הופכים שיטה חדשה לקוד שעובד בקנה מידה?	מימוש מאמרים, אימון מבוזר, אופטימיזציה

בפועל קיימת חפיפה גדולה, ושמות התפקידים משתנים בין חברות. באותה משרה בדיוק, חברה אחת תכתוב ML Engineer וחברה אחרת MLOps Engineer. קרא את תיאור המשימות, לא את הכותרת.

איך לבחור כיוון

למסלול שלך כדאי לבנות תחילה בסיס משותף של תוכנה, נתונים ו־ML, ובהמשך להעמיק באחד משני הכיוונים:

- **ML Engineer**: יותר מודלים, ניסויים, תכונות, הערכה, ניתוח טעויות וביצועי חיזוי.
- **MLOps Engineer**: יותר אוטומציה, תשתיות, אמינות, אבטחה ותפעול.

! הטעות שמעכבת הכי הרבה מתחילים

ניסיון ללמוד עשרים מוצרים במקביל. Ray, Terraform, Spark, Kafka, Kubernetes, Airflow, Kubeflow. – כל אחד מהם פותר בעיה ספציפית, ובלי להכיר את הבעיה אתה לומד תפריט פקודות ולא מקצוע. הסדר הנכון תמיד: **בעיה ← פתרון ידני ← הכלי שמייתר את העבודה הידנית.**

תרגיל 1 ▶

מצא שלוש משרות אמיתיות בתחום: אחת ל-ML Engineer, אחת ל-MLOps, ואחת ל-Data Scientist. סמן בכל אחת אילו דרישות חוזרות בשלושתן, ואילו ייחודיות. הרשימה המשותפת היא סדר העדיפות שלך לחצי השנה הראשונה.

02 מושגי המחשב הראשונים

מטרת הפרק: להבין איפה הקוד והנתונים חיים, כדי שהודעות שגיאה בהמשך יהיו מובנות ולא מפחידות.

מושג	הסבר פשוט	דוגמה בתחום
קובץ	מידע שנשמר בשם ובמיקום מסוימים	train.py, נתונים ב-CSV
תיקייה	מקום לארגון קבצים	תיקיית הפרויקט
מערכת הפעלה	התוכנה שמנהלת את המחשב	macOS, Windows, Linux
CPU	מעבד לביצוע הוראות וחישובים	הרצת Python ואימון מודל קטן
RAM	זיכרון העבודה הזמני	טבלה שטעונה בתהליך Python
דיסק	אחסון מתמשך	קובץ נתונים או מודל שנשמר
Process	תוכנית שרצה כעת	שרת החיזוי שלך
Terminal	ממשק להזנת פקודות	הפעלת סקריפט
Library	קוד מוכן שאפשר להשתמש בו	scikit-learn, pandas
API	ממשק שבאמצעותו תוכנות מתקשרות	בקשה לקבלת תחזית

ההבחנה בין RAM לדיסק חשובה מאוד ותחזור אליך בפועל: קובץ של כמה ג'יגה־בייט עשוי לצרוך הרבה יותר זיכרון כשפותחים אותו ומבצעים עליו פעולות, משום שהייצוג בזיכרון שונה מהייצוג בקובץ. יצירת עותקים של טבלה יכולה להכפיל את הצריכה בלי שתשים לב.

השגיאה שכל מתחיל פוגש

הודעה בסגנון `MemoryError` או קריסה שקטה של התהליך היא כמעט תמיד אותו סיפור: טענת לזיכרון יותר ממה שיש. הפתרון הראשון הוא לא מכונה גדולה יותר, אלא לקרוא את הנתונים בחלקים, לבחור רק את העמודות הדרושות, ולבדוק כמה עותקים אתה יוצר בדרך.

תרגיל 2

פתח תיקייה בשם `ml-learning`, צור בתוכה קובץ טקסט, וכתוב במילים שלך מה ההבדל בין הקובץ ששמרת על הדיסק לבין תוכנת העריכה שרצה בזיכרון. אם המחשב נכבה עכשיו, מה נשאר?

03 סביבת עבודה שתשרת אותך שנים

מטרת הפרק: להקים סביבת פיתוח מבודדת, ולהבין למה בידוד סביבות הוא הרגל ולא טקס.

בתחילת המסלול מספיק מחשב אישי. הפרויקטים הראשונים אינם דורשים GPU או חשבון ענן, וניסיון להתחיל משם רק יאט אותך.

התקן Python בגרסה שנתמכת בספריות שבהן תשתמש, עורך קוד כגון VS Code, ו-Git. בהמשך נוסף Docker. לדוגמאות כאן אפשר לבחור Python 3.12 ולהשתמש באותה גרסה גם בסביבת הקונטיינר.

BASH • MACOS / LINUX

```
python3 -m venv .venv
source .venv/bin/activate
```

CMD • WINDOWS

```
py -m venv .venv
.venv\Scripts\activate.bat
```

BASH • אחרי הפעלת הסביבה

```
python -m pip install --upgrade pip
python -m pip install numpy pandas matplotlib scikit-learn
```

מה בעצם עשינו

- יצר סביבת Python נפרדת לפרויקט, עם ספריות משלה. `venv`
- ההפעלה גורמת לפקודת `python` להשתמש בסביבה הזאת ולא בזו של המערכת.
- מתקין ספריות. `pip`

- מפעיל את מנהל החבילות של *Python* שבו אתה משתמש. זה מונע את התקלה הנפוצה שבה החבילה הותקנה בסביבה אחת והקוד רץ בסביבה אחרת.

HELLO.PY

```
print("My first ML engineering project")
```

BASH

```
python hello.py
```

שגיאה היא מידע, לא כישלון

אם הופיעה שגיאה, קרא אותה מהשורה האחרונה כלפי מעלה. השורה האחרונה אומרת מה קרה; השורות שמעליה אומרות איפה. עבודת פיתוח כוללת הרבה מאוד איתור תקלות, והמיומנות הזאת נמדדת בראיונות לא פחות מהיכולת לכתוב קוד.

לקורס התחלתי ללא רקע, *CS50P* של *Harvard* מתאים במיוחד. *מדריך Python הרשמי* טוב בהמשך, אבל הוא מציין בעצמו שהוא מניח היכרות בסיסית עם תכנות.

Python דרך בעיות קטנות 04

מטרת הפרק: לכתוב פונקציות קטנות, לקרוא שגיאות, ולתקן אותן בעצמך – הבסיס לכל מה שיבוא אחר כך.

תוכנית מקבלת נתונים, מבצעת עליהם פעולות, ומחזירה תוצאה. משתנה הוא שם שמפנה לערך:

PYTHON • מסתנים וסוגי נתונים

```
event_name = "Technology Conference" # str - טקסט
rooms = 4 # int - מספר שלם
hours_per_room = 1.5 # float - מספר עשרוני
has_live_stream = True # bool - אמת או שקר

estimated_hours = rooms * hours_per_room
print(estimated_hours)
```

תנאי מאפשר לבצע פעולה רק במצב מסוים:

```
if rooms > 3:
    print("Check whether another technician is needed")
else:
    print("A small setup")
```

רשימה שומרת כמה ערכים, ולולאה עוברת עליהם:

```
setup_times = [3.0, 4.5, 2.5, 6.0]

for hours in setup_times:
    print(hours)
```

פונקציה מרכזת פעולה שאפשר להפעיל שוב:

```
def average(values: list[float]) → float:
    if not values:
        raise ValueError("Cannot average an empty list")

    return sum(values) / len(values)

print(average([3.0, 4.5, 2.5, 6.0]))
```

הסימון `list[float]` מתאר איזה קלט מצופה. הוא מסייע לקורא ולכלי הפיתוח, אבל הוא **אינו אוכף לבדו** את סוגי הנתונים בזמן הריצה – Python ירוץ גם אם תעביר משהו אחר.

מילון מחבר בין שמות לערכים:

```
event = {
    "rooms": 4,
    "attendees": 350,
    "streaming": True,
}

print(event["attendees"])
print(event.get("budget", "unknown")) # לא נופל כשהמפתח חסר
```

תרגיל 3 · חמש משימות קטנות

1. כתוב פונקציה שמחזירה את זמן ההקמה הארוך ביותר מתוך רשימה.
2. ספור כמה אירועים ארכו יותר מארבע שעות.
3. חשב ממוצע רק לאירועים שכוללים שידור.
4. קבע מה התוכנית עושה כאשר חסר שדה – והחלט מה צריך לקרות.
5. החלף ערך מספרי בטקסט ובדוק איזו שגיאה מתקבלת. קרא אותה עד הסוף.

תנאי מעבר לפרק הבא

אתה מסוגל לכתוב פונקציה קטנה, לקרוא את השגיאה שלה, ולתקן אותה בלי להעתיק פתרון שלם מהאינטרנט.

05 מקוד שרץ לקוד שאפשר לתחזק

מטרת הפרק: להכיר את הכלים שהופכים סקריפט חד־פעמי למערכת שאפשר לחזור אליה בעוד חצי שנה.

סקריפט קצר יכול להיות מצוין לניסוי. מערכת שנשארת בשימוש דורשת ארגון. זה בדיוק ההבדל בין Notebook יפה לבין קוד שאפשר להעביר לייצור.

נושא	למה הוא נחוץ
Imports ו־Modules	חלוקת קוד לקבצים לפי אחריות
Exceptions	טיפול מכוון בכשלים צפויים במקום קריסה
Classes	ארגון מצב והתנהגות כשיש לכך צורך אמיתי
Generators ו־Iterators	עיבוד רצף בלי לשמור הכול בזיכרון
Type hints	הבהרת חוזים בין פונקציות
Logging	תיעוד פעולות ושגיאות בזמן ריצה
Tests	ביטחון שהשינוי לא שבר התנהגות קיימת
Configuration	שינוי הגדרות בלי לערוך את הלוגיקה
Virtual environments	הפרדת תלויות בין פרויקטים

לדוגמה, קריאת קובץ גדול שורה אחר שורה במקום טעינת הכול לזיכרון:

```
from pathlib import Path

def read_lines(path: Path):
    with path.open(encoding="utf-8") as handle:
        for line in handle:
            yield line.strip()
```

`yield` מחזיר בכל פעם פריט אחד. זה יכול לחסוך זיכרון משמעותי לעומת טעינת כל השורות לרשימה, והוא ההרגל שיציל אותך כשקובץ הנתונים יגדל.

מבני נתונים וסיבוכיות – המינימום שצריך

- מעבר על כל הרשומות: עבודה שגדלה ביחס למספר הרשומות, $O(n)$.
- השוואת כל רשומה לכל רשומה אחרת: עלולה להיות $O(n^2)$, ולהתפוצץ במאגר גדול.
- חיפוש במילון או ב-`set`: לרוב זמן ממוצע קבוע, בכפוף למימוש ולתנאים.

תרגיל 4

מצא מזהים כפולים ברשימה של מיליון רשומות באמצעות `set`. אחר כך כתוב גם את הגרסה שמשווה כל זוג רשומות, והרץ את שתיהן על 10,000 רשומות. מדוד את ההפרש והסבר אותו.

06 Linux, Git ורשתות הם חלק מהמקצוע

מטרת הפרק: להרגיש בבית בשורת הפקודה, לנהל גרסאות קוד, ולהבין איך בקשת חיזוי עוברת ברשת.

גם מודל מצוין ירוץ בסופו של דבר בתוך תהליך, על מערכת הפעלה, ויתקשר עם מערכות אחרות. מי שלא מכיר את השכבה הזאת נתקע בכל בעיה שאינה בקוד עצמו.

```
pwd                # איפה אני
ls -la             # מה יש כאן
cd project         # מעבר לתיקייה
mkdir data         # יצירת תיקייה
head -5 data/events.csv # חמש שורות ראשונות
tail -f application.log # מעקב חי אחרי לוג
grep -i "error" app.log # חיפוש בתוך קובץ
wc -l data/events.csv # ספירת שורות
du -sh artifacts/  # כמה מקום זה תופס
```

למד בהמשך גם הרשאות, משתני סביבה, תהליכים, קודי יציאה, ופלט רגיל לעומת פלט שגיאות. בכל אלה תשתמש בפועל כשתריץ שירות בקונטיינר.

מהתחלה • GIT • BASH

```
git init
git status
git add hello.py
git commit -m "Add first Python example"

git log --oneline           # מה קרה עד עכשיו
git diff                   # מה שיניתי ועוד לא נשמר
git checkout -b experiment # ענף חדש לניסוי
```

Git שומר היסטוריה של הקוד. GitHub הוא שירות שיכול לארח את המאגר ולתמוך בשיתוף פעולה ובאוטומציה. שניהם דברים שונים, ושווה לדעת מי עושה מה.

.GITIGNORE

```
.venv/
__pycache__/
.pytest_cache/
.env
artifacts/
mlruns/
mlflow.db
*.joblib
```

מה לא נכנס ל-Git ▲

סודות, מפתחות גישה, נתונים אישיים, וקבצים בינאריים כבדים. מודל שנשמר הוא תוצר, לא קוד מקור – מקומו במערכת ניהול תוצרים או באחסון אובייקטים, עם גרסה. קובץ `.env` שנדחף בטעות הוא אחד האירועים הנפוצים והיקרים ביותר בתחום.

מושג	מה זה
IP	כתובת של מכונה ברשת
DNS	תרגום שם לכתובת
Port	נקודת כניסה לשירות מסוים על אותה מכונה
HTTP	פרוטוקול לבקשות ותשובות
JSON	פורמט נפוץ להעברת נתונים
Timeout	גבול לזמן ההמתנה לתשובה
TLS	הצפנת התעבורה בין הלקוח לשרת

בקשת חיזוי היא בדרך כלל: לקוח שולח נתונים ב־HTTP, השרת מפעיל מודל, ומחזיר JSON. ברגע שתבין את המסלול הזה, גם ניפוי תקלות ייראה אחרת.

לתרגול יסודות העבודה מומלץ [Missing Semester של MIT](#).

07 SQL: לדעת להוציא את הנתונים הנכונים

מטרת הפרק: לשלוף נתונים נכונים ממסד רלציוני, ולזהות את הטעות שגורמת לרוב מאגרי האימון השבורים.

מסד נתונים רלציוני מארגן מידע בטבלאות. נניח שקיימת טבלה בשם `events`:

event_id	venue_id	attendees	setup_hours
101	A	150	3.0
102	B	400	6.0
103	A	250	4.5

שאלתה בסיסית • SQL

```
SELECT event_id, attendees, setup_hours
FROM events
WHERE attendees >= 200
ORDER BY setup_hours DESC;
```

```
SELECT
  venue_id,
  COUNT(*)      AS event_count,
  AVG(setup_hours) AS average_setup_hours
FROM events
GROUP BY venue_id;
```

```
SELECT
  e.event_id,
  e.attendees,
  v.city
FROM events AS e
JOIN venues AS v
  ON e.venue_id = v.venue_id;
```

! הטעות החשובה ביותר בפרק הזה

JOIN יכול להכפיל רשומות. אם בטבלת האולמות יש כמה שורות לכל `venue_id` – למשל שורה לכל תקופת תמחור – אירוע אחד יהפוך לכמה שורות, ומאגר האימון שלך ייתן משקל כפול לאותו אירוע בלי שתדע.

לפני ואחרי כל JOIN בדוק: מספר שורות, מספר מזהים ייחודיים, ערכים חסרים שנוצרו, והאם המפתח באמת ייחודי בצד המתאים.

```
SELECT venue_id, COUNT(*) AS rows_per_key
FROM venues
GROUP BY venue_id
HAVING COUNT(*) > 1;
```

-- יכפיל רשומות JOIN-אם חוזרות שורות, ה

למד גם `LEFT JOIN`, `CASE`, פונקציות חלון, `ROW_NUMBER`, עבודה עם תאריכים, התנהגות `NULL`, אינדקסים וטרנזקציות. פונקציות חלון בפרט הן מה שמפריד בין מי שיועד SQL בסיסי למי שיכול לבנות מאגר אימון נכון.

```
WITH ranked AS (
  SELECT
    e.*,
    ROW_NUMBER() OVER (
      PARTITION BY venue_id
      ORDER BY event_date DESC
    ) AS rn
  FROM events AS e
)
SELECT *
FROM ranked
WHERE rn ≤ 3;
```

תרגיל 5

הרץ את השאילתה למעלה, ואז הסבר במילים שלך מדוע `LIMIT 3` על כל הטבלה אינו פותר את אותה בעיה. אחר כך שנה אותה כך שתחזיר רק אולמות עם שלושה אירועים לפחות.

מדריך PostgreSQL הרשמי מתאים ללימוד מסדי נתונים ו־SQL מאפס.

08 NumPy ו־pandas: לעבוד עם נתונים בקוד

מטרת הפרק: לטעון טבלה, לבדוק אותה לפני שנוגעים בה, ולהפיק ממנה סיכומים.

NumPy מספק מערכים מספריים ופעולות יעילות עליהם. pandas מספק מבנה טבלאי בשם DataFrame.

PYTHON • NUMPY

```
import numpy as np

values = np.array([2.0, 4.0, 6.0])

print(values.mean())    # 4.0
print(values * 2)       # [ 4.  8. 12.]
print(values.shape)     # (3,)
```

מתאר את ממדי המערך. טבלה של 1,000 דוגמאות עם 20 תכונות תיוצג לעיתים כמערך בגודל `(1000, 20)`. `shape` כשמשו לא עובד ב־ML, `shape` הוא הדבר הראשון שבדקים.

```
import pandas as pd

df = pd.DataFrame({
    "venue": ["A", "B", "A"],
    "attendees": [150, 400, 250],
    "setup_hours": [3.0, 6.0, 4.5],
})

print(df.head())
print(df.dtypes)           # האם המספרים באמת מספרים
print(df.isna().sum())    # כמה חסרים בכל עמודה
print(df.describe())      # טווחים, ממוצעים וחריגים

summary = df.groupby("venue")["setup_hours"].mean()
print(summary)
```

שש השאלות לכל טבלה חדשה

1. מה מייצגת שורה אחת? אירוע? יום? לקוח?
 2. מה המשמעות והיחידות של כל עמודה?
 3. האם קיימות כפילויות, ולפי איזה מפתח?
 4. כמה ערכים חסרים יש, ולמה הם חסרים?
 5. האם התאריכים והמספרים פורשו כראוי, או נקראו כטקסט?
 6. האם הנתונים כוללים מידע שלא היה ידוע בזמן החיזוי?
- השאלה השישית תחזור לאורך כל המדריך. היא זו שמפילה מודלים בייצור.

מקורות לתרגול: NumPy למתחילים, מדריכי pandas.

המתמטיקה שבאמת צריך 09

מטרת הפרק: להבין מה החישוב עושה, כדי שתוכל לאבחן מודל שלא לומד – בלי קורס מתמטיקה מלא לפני שתתחיל.

אינך צריך לדעת הכול לפני כתיבת המודל הראשון. אתה כן צריך להבין מה קורה מתחת.

וקטור ומודל ליניארי

וקטור הוא אוסף מספרים. למשל, תיאור אירוע:

$$x = [rooms, attendees, screens]$$

$$\hat{y} = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

- x – נתוני הקלט.
- w – משקלים שהמודל לומד.
- b – איבר חופשי (bias).
- $\hat{y}$ – התחזית.

פונקציית הפסד

אם המודל חזה ארבע שעות ובפועל נדרשו שש, הוא טעה בשעתיים. פונקציית הפסד נותנת לטעות ערך מספרי שאפשר למזער:

$$L = (\hat{y} - y)^2$$

נגזרת מתארת כיצד שינוי קטן בפרמטר משפיע על התוצאה. Gradient מרכז את הנגזרות ביחס לכל הפרמטרים, ונותן את כיוון השיפור.

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \nabla L \quad \text{קצב הלמידה} = \eta$$

דוגמה פתורה, צעד אחד

חישוב מלא על נקודה אחת ✓

- נתון: $x = 2$, התשובה הנכונה $y = 5$.
- מתחילים ב- $w = 1$, $b = 0$.
- התחזית היא 2, ולכן ההפסד הוא $(2 - 5)^2 = 9$.
- הנגזרות: $\partial L / \partial w = -12$, $\partial L / \partial b = -6$.
- בקצב למידה 0.1: $w = 1 + 1.2 = 2.2$, $b = 0 + 0.6 = 0.6$.
- התחזית החדשה: $2.2 \cdot 2 + 0.6 = 5$. בדיוק.

זאת הדגמה על נקודה יחידה. במאגר אמיתי מחפשים פרמטרים שעובדים היטב על דוגמאות רבות, וגם על מידע חדש שהמודל לא ראה.

סטטיסטיקה שתשתמש בה כל שבוע

מושג	למה תזדקק לו
ממוצע וחציון	חציון עמיד לחריגים; ממוצע לא. זה משנה כשיש אירוע אחד בן 40 שעות.
שונות וסטיית תקן	כמה הערכים מפוזרים סביב המרכז
אחוזונים	מדידת זמני תגובה: p95 חשוב יותר מהממוצע
הסתברות מותנית	הבסיס לסיווג ולחשיבה על מדדים
התפלגויות ודגימה	להבין מה מייצג המדגם שלך, ומה לא
מתאם מול סיבתיות	ההבדל בין חיזוי טוב להסקה שגויה

למידת מכונה

עכשיו לומדים מה מודל עושה, איך מגדירים בעיה שאפשר לפתור, איך בונים נתונים שאפשר לסמוך עליהם, ואיך יודעים אם התוצאה טובה באמת.

מה המודל באמת לומד 10

מטרת הפרק: להחזיק אוצר מילים מדויק, ולהבחין בין אימון לשימוש.

למידת מכונה היא התאמת פונקציה לדוגמאות, לפי מטרה מוגדרת.

מושג	משמעות
Sample	דוגמה אחת
Feature	תכונה שמשמשת כקלט
Label / Target	התשובה שרוצים ללמוד לחזות
Training	התאמת המודל לנתונים
Inference	שימוש במודל שכבר אומן
Parameter	ערך שנלמד באימון
Hyperparameter	הגדרה של תהליך הלמידה או של מבנה המודל
Generalization	ביצועים על דוגמאות חדשות

UNSUPERVISED

למידה לא מונחית

מחפשים מבנה בלי תשובת יעד: קיבוץ לקוחות, זיהוי חריגות, הקטנת ממדים.

SUPERVISED

למידה מונחית

לומדים מדוגמאות עם תשובות: סיווג פניות, חיזוי זמן הקמה, זיהוי הונאה. כאן תתחיל.

REINFORCEMENT

למידת חיזוק

לומדים פעולה מתוך אינטראקציה עם סביבה ותגמולים. תחום מרתק, אבל לא נקודת פתיחה.

בתוך למידה מונחית, שתי משימות עיקריות: **Regression** מחזירה ערך מספרי, כגון זמן הקמה. **Classification** מחזירה מחלקה או הסתברויות למחלקות, כגון סוג פנייה.

אימון ושימוש הם שלבים נפרדים

אין צורך לאמן מחדש בכל בקשת חיזוי. האימון קורה פעם אחת (או מדי תקופה), מייצר תוצר, והשירות טוען את התוצר הזה. בלבול בין השניים הוא מקור נפוץ לארכיטקטורות שגויות ולשאלות ראיון שנופלות.

להרחבה מסודרת: [Google Machine Learning Crash Course](#).

11 להתחיל מהבעיה, לא מהאלגוריתם

מטרת הפרק: לכתוב מסמך בן עמוד אחד שמגדיר מה המערכת אמורה לעשות – לפני שורת קוד אחת.

לפני בחירת אלגוריתם, כתוב משפט אחד שמגדיר את המערכת:

הגדרה לדוגמה

בכל בוקר בשעה 06:00, המערכת תעריך את זמן ההקמה של האירועים המתוכננים ל-72 השעות הקרובות, כדי לעזור למנהל התפעול לשבץ צוותים.

כעת שאל את השאלות שקובעות אם הפרויקט יצליח:

1 באיזה רגע מופקת התחזית?

זה קובע אילו נתונים בכלל קיימים.

2 אילו נתונים כבר ידועים באותו רגע?

כל השאר אסור לשימוש.

3 מה בדיוק פירוש "זמן הקמה"?

מהגעת המשאית? מכניסת הצוות? עד מתי?

4 מה המחיר של הערכת חסר לעומת הערכת יתר?

לרוב הם לא סימטריים.

5 מי משתמש בתוצאה, ואיך?

מספר במסך? החלטה אוטומטית? המלצה?

6 מה עושים כאשר חסר מידע?

לא לחזות? לחזות עם ברירת מחדל?

7 מה ה-Baseline?

מה הפתרון הפשוט ביותר שאפשר להשוות אליו?

Baseline: הדבר שאתה חייב לנצח

הפתרון הפשוט הוא ה-Baseline. למשל, חיזוי חציון זמן ההקמה לפי סוג אירוע. אם מודל מורכב אינו משפר את התוצאה בצורה משמעותית, ייתכן שהמורכבות אינה מוצדקת – וזה מסקנה לגיטימית שמעידה על בגרות מקצועית, לא על כישלון.

! חיזוי אינו סיבתי

העובדה שבנתונים אירועים עם יותר טכנאים נמשכו יותר זמן אינה מוכיחה שטכנאים מאריכים את ההקמה. סביר יותר שפשוט שולחים יותר טכנאים לאירועים מורכבים. מודל חיזוי ילמד את הקשר הזה וישתמש בו בהצלחה – אבל אם משהו יסיק ממנו "נצמצם טכנאים כדי לקצר הקמה", המערכת שלך תגרום נזק.

תוצר הפרק ▶

עמוד אחד שמגדיר: המטרה, רגע החיזוי, הנתונים הזמינים, המדד, ה-Baseline, ומה קורה כשאינ מספיק מידע. בלי המסמך הזה, כל שיפור מודל הוא ניחוש.

12 נתונים שאפשר לסמוך עליהם

מטרת הפרק: לתעד תכונות כמו מהנדס, ולזהות דליפת מידע לפני שהיא הורסת את הפרויקט.

רבות מהבעיות במערכות ML מתחילות לא באלגוריתם, אלא בהגדרת הנתונים. עבור כל תכונה תעד:

שאלה	דוגמה
מה המשמעות?	מספר משתתפים מתוכנן
מה היחידה?	אנשים
מתי הערך נוצר?	במועד אישור ההזמנה
מתי הוא זמין למערכת?	אחרי סנכרון לילי ממערכת ההזמנות
מה עושים כשחסר?	דחייה, השלמה, או ערך חסר מפורש
האם ההגדרה משתנה?	מתוכנן לעומת בפועל – לא אותו שדה

Data leakage: הכשל השקט

Data leakage מתרחש כאשר תהליך הלמידה משתמש במידע שלא היה זמין במצב החיזוי האמיתי. התוצאה: ציונים מצוינים באימון, וכישלון מוחלט בייצור.

! ארבע צורות דליפה שתפגוש בפועל

1. **מידע מהעתיד:** חיזוי זמן הקמה באמצעות מספר שעות העבודה שחויבו אחרי האירוע.
2. **חלוקה שגויה:** רשומות מאותו אירוע מופיעות גם באימון וגם בבדיקה.
3. **עיבוד לפני חלוקה:** חישוב ממוצעי השלמה או Scaling על כל המאגר לפני הפיצול.
4. **עדכון בדיעבד:** שימוש בשדה עסקי שעודכן לאחר מעשה, כאילו היה ידוע בזמן אמת.

יש גם הבדל קריטי בין **מועד האירוע** לבין **מועד הגעת המידע**. רשומה המתארת את יום שני, אך הגיעה למערכת ביום רביעי, לא הייתה זמינה לחיזוי ביום שלישי. מאגר אימון שמתעלם מזה נראה מצוין ומשקר.

PIPELINE-העיבוד חייב לשבת בתוך ה • PYTHON

```
# ראה גם את נתוני הבדיקה Scaler-לא נכון - ה
scaler = StandardScaler().fit(X) # דליפה ←
X_train, X_test = train_test_split(scaler.transform(X))

# מחשב את העיבוד מחדש רק על נתוני האימון שלו Fold נכון - כל
pipeline = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression()),
])
cross_val_score(pipeline, X_train, y_train, cv=5)
```

חלוקה נכונה ו-Pipeline מצמצמים טעויות הכנה, אבל **אינם מזהים אוטומטית** תכונה עסקית שמכילה מידע עתידי. את זה רק אתה יכול לתפוס, בשיחה עם מי שמכיר את הנתונים. ראה גם [הנחיות scikit-learn למניעת דליפה](#).

קח מאגר כלשהו שאתה מכיר, ועבור על כל עמודה עם השאלה: "האם הערך הזה יהיה ידוע ברגע שבו אני מתיימר לחזות?". מצא לפחות עמודה אחת שהתשובה לגביה אינה חד־משמעית, וכתוב מה תעשה איתה.

13 להפריד אימון, בחירת מודל ובדיקה

מטרת הפרק: לבחור חלוקת נתונים שמשקפת את השימוש האמיתי במערכת.

חלק	תפקיד	כמה פעמים מסתכלים עליו
Train	לימוד הפרמטרים	כל הזמן
Validation	בחירת מודל, הגדרות וסף החלטה	הרבה
Test	הערכה סופית אחרי שכל הבחירות נסגרו	פעם אחת

אין יחס חלוקה יחיד שמתאים לכל בעיה. 80/20 הוא מוסכמה נוחה, לא חוק. בחר את שיטת החלוקה לפי אופן השימוש:

מה המערכת תעשה בפועל	איך לחלק
תחזיות לעתיד	חלוקה לפי זמן – אימון על עבר, בדיקה על תקופה מאוחרת
לשרת לקוחות חדשים	הפרדה בין לקוחות – לקוח שלם באימון או שלם בבדיקה
כמה תמונות מאותו אדם	הפרדה לפי אדם
דוגמאות עצמאיות יחסית	לעיתים חלוקה אקראית מספיקה

Cross-validation חוזרת על האימון וההערכה במספר חלוקות. היא מסייעת להשוות מועמדים ולראות עד כמה התוצאה יציבה – הפרש של 1% בין שני מודלים כשסטיית התקן בין ה־Folds היא 4% אינו הפרש.

גם סט הבדיקה נשחק

סט בדיקה "דלף" לתהליך אם הסתכלת עליו שוב ושוב ובחרת שינויים לפי התוצאה שלו. אחרי עשרים החלטות כאלה, הוא כבר לא מודד הכללה – הוא מודד כמה טוב התאמת את עצמך אליו. החזק אותו סגור.

בנתוני זמן, שמור על כיוון הזמן. אין לערבב את העתיד לתוך האימון, גם לא דרך Cross-validation אקראית.

יש לך מאה לקוחות ועשרים רשומות מכל לקוח. הסבר למה חלוקה אקראית של השורות תציג תמונה אופטימית מדי כשמטרתך לשרת לקוחות חדשים. אחר כך ממש חלוקה נכונה עם `GroupKFold`.

14 לבחור אלגוריתם מתוך הבנה

מטרת הפרק: להכיר מספר משפחות אלגוריתמים לעומק, ולדעת מה לבדוק בכל אחת.

משפחה	הרעיון	מה חשוב לבדוק
Linear Regression	שילוב משוקלל של תכונות	קשרים לא ליניאריים וחריגים
Logistic Regression	מודל ליניארי לסיווג הסתברותי	Scaling, רגולריזציה וכיוול
Decision Tree	רצף פיצולים לפי תנאים	עץ עמוק ישנן את הנתונים
Random Forest	שילוב עצים שנלמדו עם אקראיות	גודל, זמן חיזוי וביצועים
Gradient Boosting	הוספת מודלים שמתקנים שגיאות	כוונון, עצירה מוקדמת ורגולריזציה
k-NN	הסתמכות על דוגמאות קרובות	Scaling ועלות חיפוש
Naive Bayes	מודל הסתברותי עם הנחות פשטניות	האם ההנחות מתאימות לבעיה
SVM	מציאת גבול החלטה עם מרווח	Scaling ועלות במאגרים גדולים
Neural Networks	שכבות של טרנספורמציות נלמדות	כמות נתונים, חישוב ותהליך אימון

✓ מה לבחור בפועל, לנתונים טבלאיים

התחל ב־Baseline פשוט, אחר כך Logistic או Linear Regression בתוך Pipeline, ואז Gradient Boosting. ברוב בעיות הטבלה בעולם האמיתי, Gradient Boosting מכון היטב הוא נקודת ההשוואה שקשה לנצח – גם מול רשתות עצביות.

Underfitting ו־Overfitting

- **Overfitting**: הצלחה על נתוני האימון שאינה נשמרת על מידע חדש. המודל שינן רעש.
- **Underfitting**: המודל אינו תופס מספיק מהמבנה הרלוונטי. גם האימון וגם הבדיקה חלשים.

דרכי טיפול אפשריות: שיפור איכות הנתונים, שינוי תכונות, הגבלת מורכבות, רגולריזציה, Early stopping, ואיסוף נתונים נוספים שמתאימים לבעיה.

▲ אל תאבחן לפי הפער בלבד

פער גדול בין Train ל־Validation מרמז על Overfitting, אבל שני ציונים חלשים ודומים מרמזים על בעיה אחרת לגמרי – ואולי על כך שהתכונות פשוט לא מכילות את המידע הדרוש. הסתכל על שני המספרים, לא על ההפרש.

מטרת הפרק: לבחור מדד שמייצג את ההחלטה העסקית, ולהסביר אותו לאדם שאינו טכני.

בסיווג בינארי, הגדר קודם איזו מחלקה היא "חיובית". נניח שהמודל מזהה אירועים עם סיכון לתקלה:

ללא תקלה	תקלה בפועל	
FP = 20	TP = 80	המודל התריע
TN = 860	FN = 40	המודל לא התריע

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 80 / 100 = 80\%$$

מתוך ההתרעות שהופקו, 80% היו מוצדקות.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 80 / 120 \approx 66.7\%$$

זיהינו כשני שלישים מהתקלות שקרו בפועל.

$$\text{F1} = 2 \cdot \text{TP} / (2 \cdot \text{TP} + \text{FP} + \text{FN}) \approx 72.7\%$$

$$\text{Accuracy} = (\text{TP} + \text{TN}) / N = 940 / 1000 = 94\%$$

! למה 94% כאן הוא מספר חסר משמעות

מודל שפשוט יגיד "אין תקלה" תמיד, יקבל Accuracy 88% על אותם נתונים – בלי לזהות ולו תקלה אחת. במאגר לא מאוזן, Accuracy מודד בעיקר את גודל המחלקה הגדולה.

לתרגם טעויות לכסף

המדד הנכון תלוי בהחלטה העסקית. אם פספוס תקלה עולה 20 יחידות והתרעת שווא עולה יחידה אחת:

$$\text{עלות} = 1 \times 20 + 20 \times 40 = 820$$

זהו חישוב לימודי, לא תמחור של מערכת אמיתית. אבל המבנה הזה – מטריצת עלויות שמכפילה את מטריצת הבלבול – הוא איך שבוחרים סף החלטה בעבודה אמיתית.

מדד	מתי להשתמש
MAE	טעות מוחלטת ממוצעת, ביחידות היעד. קל להסביר: "בממוצע אנחנו מפספסים ב־1.2 שעות".
RMSE	נותן משקל חזק יותר לטעויות גדולות. בחר בו כשטעות גדולה אחת גרועה משתי טעויות קטנות.
MAPE	טעות באחוזים. נזהר ממנו כשהיעד יכול להתקרב לאפס.
Quantile loss	כאשר רוצים לחזות אחוזון – למשל זמן שיספיק ב־90% מהמקרים.

מה עוד ללמוד

- Precision–Recall curves, ולמה הן עדיפות על ROC במאגרים לא מאוזנים.
- ROC-AUC והגבולות שלו.
- Macro לעומת Weighted averaging בסיווג רב־מחלקתי.
- **Calibration**: האם תחזיות של 80% אכן מתרחשות בכ־80% מהמקרים הדומים? מודל מדויק יכול להיות לא מכויל, וזה קריטי כשהתחזית מזינה החלטה כלכלית.
- **מדדים לפי קבוצות**, ולא רק ממוצע כללי. מודל עם 92% ממוצע יכול להיות 60% על פלח לקוחות שלם.

תיעוד מדדי [scikit-learn](#).

16 Feature engineering הוא תכנון מידע

מטרת הפרק: לתכנן תכונות שמייצגות משהו אמיתי, וזמינות ברגע החיזוי.
תכונה טובה מייצגת משהו שרלוונטי לבעיה, וזמין בזמן החיזוי. שני התנאים חובה.

היסטוריה

ממוצע הקמה קודם

ממוצע זמן ההקמה באירועים *קודמים בלבד* באותו אולם, עם גבול זמן מפורש.

יחס

מסכים לכל חדר

יחסים לרוב מכלילים טוב יותר מערכים גולמיים.

גיוון

מספר סוגי ציוד

מייצג מורכבות תפעולית שלא נראית במספר הפריטים.

זמן

יום בשבוע ושעות עד האירוע

דפוסים מחזוריים ולחץ זמן – שניהם משפיעים בפועל.

בעיה	טיפול אפשרי
ערכים חסרים	Imputation, או מודל שתומך בחסרים ישירות
גדלים בסולמות שונים	Scaling – הכרחי ל-k-NN, SVM ורשתות
קטגוריות	One-hot encoding, או קידוד מתאים אחר
קטגוריות עם ערכים רבים	Target encoding בזהירות – הוא מקור דליפה קלאסי
טקסט	TF-IDF או Embeddings
ערכים קיצוניים	בדיקת מקורם וטיפול לפי המשמעות
מידע בזמן	תכונות היסטוריות עם גבול זמן מפורש

שתי החלטות שנראות טכניות ואינן

אין למחוק חריג רק משום שהוא נדיר. אולי הוא שגיאת הקלדה, ואולי זה המקרה העסקי החשוב ביותר שאתה אמור לזהות.

השלמת חוסרים היא החלטה. "לא ידוע" אינו שקול לאפס. לפעמים העובדה שערך חסר היא בעצמה התכונה החזקה ביותר – שווה להוסיף דגל `is_missing` ולבדוק.

תרגיל 8

תכנן חמש תכונות לחיזוי זמן הקמה. לכל אחת כתוב שלוש שורות: למה היא רלוונטית, מתי בדיוק היא זמינה, ואיך היא עלולה להדליף מידע. תכונה שאינך יכול לענות עליה – לא נכנסת.

17 ניתוח טעויות: איך משפרים מודל בשיטתיות

מטרת הפרק: להפסיק לנחש מה לשפר, ולהתחיל לקרוא את הטעויות של המודל.

זה הפרק שמפריד בין מי שמריץ מודלים לבין מי שבונה מערכות. אחרי שיש לך מודל ראשון, השאלה אינה "איזה אלגוריתם לנסות עכשיו", אלא "על מה בדיוק הוא טועה, ולמה".

התהליך

1 אסוף את הטעויות הגרועות ביותר.

מיין את דוגמאות הוולידציה לפי גודל הטעות, וקח את 50 העליונות.

2 קרא אותן אחת־אחת.

כן, ידנית. אין תחליף. חפש מה משותף להן.

3 תייג את הטעויות לקטגוריות.

למשל: "תווית שגויה במקור", "אירוע מסוג שלא ראינו", "ערך חסר", "מקרה קצה אמיתי".

4 ספור כל קטגוריה.

4

עכשיו יש לך סדר עדיפויות מבוסס נתונים במקום תחושה.

5 העריך תקרת שיפור.

5

אם 60% מהטעויות הן תיוג שגוי במקור, שום אלגוריתם לא יעזור – צריך לתקן את הנתונים.

6 בחר פעולה אחת

6

, בצע, ומדוד מחדש את אותו פילוח.

פילוח טעויות לפי קבוצה • PYTHON

```
import pandas as pd

results = X_val.copy()
results["actual"] = y_val
results["predicted"] = model.predict(X_val)
results["error"] = (results["predicted"] - results["actual"]).abs()

# איפה המודל הכי גרוע?
by_segment = (
    results
    .groupby("venue_type")
    .agg(rows=("error", "size"), mean_error=("error", "mean"))
    .sort_values("mean_error", ascending=False)
)
print(by_segment)

# חמישים הטעויות הגדולות ביותר, לקריאה ידנית
worst = results.nlargest(50, "error")
worst.to_csv("worst_errors.csv", index=False)
```

שלוש שאלות שמניבות את רוב השיפור



1. האם יש פלח שבו המודל גרוע במיוחד? לרוב כן. ולרוב יש לזה סיבה שאפשר לתקן.
2. האם התוויות עצמן נכונות? בדוק 30 דוגמאות ידנית. אחוז תיוג שגוי הוא תקרת הביצועים שלך.
3. האם המודל טועה יותר בדוגמאות חדשות מבחינת זמן? אם כן, יש שינוי בהתפלגות שכדאי להבין לפני שמוסיפים מורכבות.

עקומת למידה: יותר נתונים או מודל אחר?

אמן את אותו מודל על 10%, 25%, 50% ו-100% מהנתונים, ומדוד. אם הציון עדיין עולה בקצה – יותר נתונים יעזרו. אם הוא השתטח – יותר נתונים מאותו סוג לא יעזרו, וצריך תכונות טובות יותר או מודל אחר. זו בדיקה של עשר דקות שחוסכת שבועות.

על המודל מפרק 18, הפק קובץ של 30 הטעויות הגרועות ביותר, תייג אותן לשלוש קטגוריות לפחות, וכתוב פסקה אחת: מה הפעולה הבאה שלך ולמה דווקא היא.

מִNotebook לשירות

כאן מתחילה ההנדסה. מודל שיושב במחברת אינו מוצר. בחלק הזה נהפוך אותו לשירות עם חוזה, בדיקות, גרסאות, קונטיינר ותהליך שחרור.

פרויקט ראשון: אימון ושמירת מודל 18

מטרת הפרק: להריץ תהליך אימון שלם שמייצר תוצרים ניתנים לשחזור – ולהבין כל החלטה בו.

הקוד הזה קיים ורץ

כל הקבצים בפרקים 18–24 נמצאים בתיקייה `labs/iris-service` במאגר הזה, כולל `Makefile` ובדיקות. הם הורצו ונבדקו, לא רק נכתבו.

נשתמש ב־`iris`, מאגר קטן של מדידות פרחים שמגיע עם `scikit-learn`. הוא מתאים ללימוד התהליך; הוא **אינו** פרויקט מספק לתיק עבודות בפני עצמו.

התקנה · BASH

```
python -m pip install scikit-learn pandas numpy joblib
python -m pip install fastapi uvicorn pytest httpx
```

```

from pathlib import Path
import hashlib
import json
import platform
from uuid import uuid4

import joblib
import pandas as pd
import sklearn

from sklearn.datasets import load_iris
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.model_selection import (
    GridSearchCV,
    StratifiedKFold,
    cross_val_score,
    train_test_split,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = [
    "sepal_length_cm",
    "sepal_width_cm",
    "petal_length_cm",
    "petal_width_cm",
]

def main():
    dataset = load_iris()

    X = pd.DataFrame(dataset.data, columns=FEATURES)
    y = dataset.target

    # טי הבדיקה נשמר בצד לפני כל החלטה על הגדרות
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=11,
    )

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=11)

    baseline_scores = cross_val_score(
        DummyClassifier(strategy="most_frequent"),
        X_train, y_train, scoring="f1_macro", cv=cv,
    )

    pipeline = Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=2000)),
    ])

    search = GridSearchCV(

```

```

    estimator=pipeline,
    param_grid={"model__C": [0.1, 1.0, 10.0]},
    scoring="f1_macro",
    cv=cv,
    n_jobs=1,
    refit=True,
)

search.fit(X_train, y_train)
model = search.best_estimator_

# הערכה סופית, אחרי שכל ההגדרות כבר נבחרו
predictions = model.predict(X_test)
test_f1 = f1_score(y_test, predictions, average="macro")

dataset_hash = hashlib.sha256(
    dataset.data.tobytes() + dataset.target.tobytes()
).hexdigest()

metadata = {
    "model_version": uuid4().hex,
    "features": FEATURES,
    "target_names": dataset.target_names.tolist(),
    "python_version": platform.python_version(),
    "sklearn_version": sklearn.__version__,
    "dataset_sha256": dataset_hash,
    "split_seed": 11,
    "best_parameters": search.best_params_,
}

metrics = {
    "baseline_cv_macro_f1": float(baseline_scores.mean()),
    "selected_cv_macro_f1": float(search.best_score_),
    "test_macro_f1": float(test_f1),
}

artifact_dir = Path(__file__).resolve().parent / "artifacts"
artifact_dir.mkdir(exist_ok=True)

joblib.dump(
    {"pipeline": model, "metadata": metadata},
    artifact_dir / "model.joblib",
)
(artifact_dir / "metrics.json").write_text(
    json.dumps(metrics, indent=2), encoding="utf-8",
)
(artifact_dir / "metadata.json").write_text(
    json.dumps(metadata, indent=2), encoding="utf-8",
)

print(json.dumps(metrics, indent=2))

if __name__ == "__main__":
    main()

```

```
python train.py
```

הקוד יוצר שלושה קבצים בתיקיית `artifacts`. אין כאן ציון שהומצא מראש – התוצאה תחושב בסביבה שלך.

◆ חמש ההחלטות שחשוב להבין בקוד הזה

1. ה־Test נשמר **מחוץ** לחיפוש ההגדרות, ונבדק פעם אחת בסוף.
2. ה־Scaling נמצא **בתוך** ה־Pipeline, ולכן נלמד מחדש בכל Fold – בלי דליפה.
3. ה־Baseline נמדד על אותן חלוקות אימון בדיוק, אחרת ההשוואה חסרת משמעות.
4. נשמרים גם סדר התכונות וגם גרסאות הספריות – בלעדיהם אי אפשר יהיה לטעון את המודל בעוד שנה.
5. החלוקה האקראית היא בחירה *לדוגמת הלימוד הזאת*, לא תבנית אוטומטית לכל מאגר (ראה פרק 13).

מבצע חיפוש על שילובי ההגדרות, ובברירת המחדל `refit=True` מאמן מחדש את המועמד שנבחר על כל נתוני האימון שסופקו לו (תיעוד).

! אזהרת אבטחה על קובצי מודל

קובצי `joblib` מבוססים על מנגנוני `pickle`. טעינת קובץ כזה יכולה להריץ קוד שרירותי. טען רק ממקור מהימן, ושמור התאמה בין סביבת האימון לסביבת הטעינה. ראה *שמירת מודלים ב-scikit-learn*.

19 שירות חיזוי עם חוזה ברור

מטרת הפרק: להפוך מודל שמור לשירות שמגדיר בדיוק מה הוא מקבל ומה הוא מחזיר.

ה־API הוא החוזה בין המודל לשאר העולם. חוזה רופף הוא מקור מובטח לתקלות בייצור.

```

from contextlib import asynccontextmanager
from pathlib import Path
from typing import Any
from uuid import uuid4

import joblib
import pandas as pd

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, ConfigDict, Field

ARTIFACT = Path(__file__).resolve().parent / "artifacts" / "model.joblib"

@asynccontextmanager
async def lifespan(app: FastAPI):
    # המודל נטען פעם אחת בעליית השירות, ולא בכל בקשה
    app.state.bundle = joblib.load(ARTIFACT)
    app.state.ready = True
    yield
    app.state.ready = False

app = FastAPI(lifespan=lifespan)

class Flower(BaseModel):
    model_config = ConfigDict(extra="forbid", allow_inf_nan=False)

    sepal_length_cm: float = Field(gt=0, le=20)
    sepal_width_cm: float = Field(gt=0, le=20)
    petal_length_cm: float = Field(gt=0, le=20)
    petal_width_cm: float = Field(gt=0, le=20)

@app.get("/health")
def health():
    """Liveness: האם התהליך חי."""
    return {"status": "ok"}

@app.get("/ready")
def ready():
    """Readiness: האם אפשר לשלוח לכאן תעבורה."""
    if not getattr(app.state, "ready", False):
        raise HTTPException(status_code=503, detail="model is not loaded")
    return {"status": "ready"}

@app.post("/predict")
def predict(flower: Flower) → dict[str, Any]:
    bundle = app.state.bundle
    model = bundle["pipeline"]
    meta = bundle["metadata"]

```

```
# סדר העמודות נלקח מהמטא-דאטה, לא מסדר השדות בבקשה
row = pd.DataFrame([flower.model_dump()], columns=meta["features"])

class_id = int(model.predict(row)[0])
probabilities = model.predict_proba(row)[0]

class_probabilities = {
    meta["target_names"][int(label)]: float(probability)
    for label, probability in zip(model.classes_, probabilities)
}

return {
    "prediction_id": uuid4().hex,
    "prediction": meta["target_names"][class_id],
    "probabilities": class_probabilities,
    "model_version": meta["model_version"],
}
```

הפעלה ובדיקה • BASH

```
python -m uvicorn app:app --host 127.0.0.1 --port 8000
```

בטרמינל אחר:

```
curl -X POST http://127.0.0.1:8000/predict \
-H 'Content-Type: application/json' \
-d '{"sepal_length_cm":5.1,"sepal_width_cm":3.5,
    "petal_length_cm":1.4,"petal_width_cm":0.2}'
```

פתח בדפדפן את ממשק התיעוד המקומי בכתובת <http://127.0.0.1:8000/docs> ושולח בקשה משם.

ארבע החלטות הנדסיות בקוד הזה

1. המודל נטען בעליית השירות, לא בכל בקשה. זו הסיבה לשימוש במנגנון `lifespan` (תיעוד FastAPI).
2. דוחה שדה לא מוכר במקום להתעלם ממנו בשקט. שדה שהשתנה בצד הלקוח יתגלה מיד, לא אחרי שבועיים של תחזיות גרועות. `extra="forbid"`
3. סדר העמודות מגיע מהמטא-דאטה. זו הגנה מפני אחת התקלות היקרות ביותר: שתי תכונות שהתחלפו במקומן.
4. `prediction_id` מוחזר ללקוח. בלעדיו לא תוכל לחבר תחזית לתוצאה שקרתה בפועל – וזה מה שיאפשר למדוד איכות בייצור (פרק 29).

גבול של 20 ס"מ הוא חוזה לימודי שבחרנו לדוגמה. במערכת אמיתית ההגבלות נקבעות לפי משמעות הנתונים ודרישות המוצר.

הסתברות שהמודל מחזיר אינה הבטחת נכונות, במיוחד כאשר הקלט שונה מהותית מהמידע שעליו אומן. מודל יכול להיות בטוח מאוד וטועה לחלוטין.

לבדוק תוכנה, נתונים ומודל בנפרד 20

מטרת הפרק: להבין שבדיקות במערכת ML הן לא סוג אחד, ולכתוב בדיקות שבאמת תופסות תקלות.

שכבה	דוגמה לבדיקה
קוד	פונקציה מחזירה תוצאה צפויה
נתונים	שדה חובה קיים, וערכים בטווח סביר
הכנת נתונים	סדר התכונות נשמר לאורך ה-Pipeline
מודל	ביצועים על נתונים מתאימים, ביחס ל-Baseline
API	קלט לא תקין נדחה עם קוד השגיאה הנכון
אינטגרציה	מודל שנשמר נטען ומחזיר תוצאה
תפעול	השירות עולה, עומד בעומס, וניתן להחזיר גרסה

```

import json
import math
from pathlib import Path

import pytest
from fastapi.testclient import TestClient

from app import app

VALID_INPUT = {
    "sepal_length_cm": 5.1,
    "sepal_width_cm": 3.5,
    "petal_length_cm": 1.4,
    "petal_width_cm": 0.2,
}

ARTIFACTS = Path(__file__).resolve().parent / "artifacts"

@pytest.fixture(scope="module")
def client():
    with TestClient(app) as test_client:
        yield test_client

def test_prediction_contract(client):
    response = client.post("/predict", json=VALID_INPUT)
    assert response.status_code == 200

    body = response.json()
    assert body["prediction"] in {"setosa", "versicolor", "virginica"}

    probabilities = list(body["probabilities"].values())
    assert len(probabilities) == 3
    assert all(0 ≤ value ≤ 1 for value in probabilities)
    assert math.isclose(sum(probabilities), 1.0, abs_tol=1e-6)
    assert body["model_version"]

def test_invalid_measurement_is_rejected(client):
    invalid = {**VALID_INPUT, "petal_length_cm": -1}
    assert client.post("/predict", json=invalid).status_code == 422

def test_extra_field_is_rejected(client):
    invalid = {**VALID_INPUT, "unexpected_field": 123}
    assert client.post("/predict", json=invalid).status_code == 422

def test_feature_order_is_preserved(client):
    """אותן מדידות בסדר שדות אחר חייבות להחזיר אותה תחזית."""
    reversed_input = dict(reversed(list(VALID_INPUT.items())))
    first = client.post("/predict", json=VALID_INPUT).json()
    second = client.post("/predict", json=reversed_input).json()
    assert first["prediction"] == second["prediction"]

```

```
def test_model_beats_baseline():
    """בדיקת מודל, לא בדיקת קוד: התוצאה חייבת להצדיק את המורכבות"""
    metrics = json.loads((ARTIFACTS / "metrics.json").read_text())
    gap = metrics["selected_cv_macro_f1"] - metrics["baseline_cv_macro_f1"]
    assert gap > 0.20
    assert metrics["test_macro_f1"] ≥ 0.85
```

BASH

```
python train.py      # קודם יוצרים מודל
python -m pytest -q  # ואז בודקים
```

שתי הבדיקות ששווה להעתיק לכל פרויקט

תופס את התקלה שבה מישהו שינה את סדר השדות בצד `test_feature_order_is_preserved`
הלוקח. הופך את "המודל מספיק טוב" מדעה לשער אוטומטי שמונע `test_model_beats_baseline`.
שחרור של מודל גרוע.

בדיקות ב-FastAPI.

21 להפוך ניסויים לתהליך מדעי

מטרת הפרק: לנהל ניסויים כך שתוכל להשוות ביניהם בעוד חודשיים, ולא רק לזכור שמשוהו עבד.

לכל ניסוי צריכים להיות שינוי מוגדר והשערה.

ניסוי חלש

אין מה ללמוד

"שיניתי כמה דברים והציון עלה." מה בדיוק עזר? אי אפשר לדעת, ואי אפשר לחזור על זה.

ניסוי טוב

יש השערה

"ייתכן שסוג האולם משפר חיזוי משום שהוא מייצג הבדלים במורכבות ההקמה."

לכל ניסוי שמור: גרסת הקוד, גרסת הנתונים, הגדרות החלוקה, התכונות והכנת הנתונים, האלגוריתם וההגדרות, המדדים, משך האימון, גודל המודל, והמסקנה.

```
import json
from pathlib import Path

import mlflow

root = Path(__file__).resolve().parent
metrics = json.loads((root / "artifacts" / "metrics.json").read_text())
metadata = json.loads((root / "artifacts" / "metadata.json").read_text())

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("iris-learning")

with mlflow.start_run():
    mlflow.log_params({
        "split_seed": metadata["split_seed"],
        "model_version": metadata["model_version"],
        "model_C": metadata["best_parameters"]["model__C"],
    })
    mlflow.set_tag("dataset_sha256", metadata["dataset_sha256"])
    mlflow.log_metrics(metrics)
    mlflow.log_artifacts(str(root / "artifacts"))
```

```
python -m pip install mlflow
python track_run.py
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
```

Registry: זהות וגרסאות של מודל

הדוגמה שומרת קבצים כתוצרי ניסוי. השלב הבא הוא שימוש בפורמט מודל של MLflow וב־Model Registry, שמנהל זהות וגרסאות.

Alias אינו פריסה

Alias כגון `champion` הוא הפניה ניתנת לשינוי לגרסה מסוימת. שינוי ה־Alias **אינו מבטיח** שתהליך שרת שכבר טען מודל לזיכרון ירענן אותו אוטומטית. מי שמניח אחרת מגלה בייצור שהוא עדיין מגיש את הגרסה הישנה. ראה [MLflow Model Registry](#).

שחזוריות: להוכיח מאיפה הגיעה התחזית 22

מטרת הפרק: להבין מה באמת נדרש כדי לשחזר תוצאה, ולמה `seed` לבדו לא מספיק.

השירות הוא רק חלק מהמערכת. כדי לשחזר תחזית צריך לדעת הרבה יותר מ"איזה קובץ מודל נטען":

- גרסת הקוד והספריות.

- גרסת הכנת הנתונים.
- סדר התכונות והיחידות.
- הגדרות המודל.
- גרסת הנתונים והחלוקה.
- סף ההחלטה.
- סביבת הריצה עצמה.

Seed אינו ערובה !

מסייע לשלוט באקראיות, אבל אינו מבטיח לבדו תוצאות זהות בין חומרות שונות, `random_state` גרסאות ספריות שונות, או אלגוריתמים מקביליים. שחזוריות היא תוצאה של תיעוד ונעילת סביבה, לא של מספר אחד בקוד.

תרגיל 10 • מבחן השחזור

מחק את הסביבה הווירטואלית של פרויקט הלימוד. בנה אותה מחדש מאפס, והרץ שוב את האימון. תעד כל דבר שהיה חסר לך כדי להצליח – הרשימה הזאת היא בדיוק מה שצריך להיכנס ל-README ולקובץ התלויות.

Docker: לארוז סביבת ריצה 23

מטרת הפרק: להפוך "זה עובד אצלי" לתוצר שרץ אותו בכל מקום.

Docker Image הוא תוצר שמכיל את האפליקציה ואת סביבת המשתמש הדרושה לה. Container הוא מופע שרץ מתוך ה־Image. שניהם דברים שונים, וההבחנה נשאלת בראיונות.

REQUIREMENTS.TXT

```
scikit-learn=1.9.1
pandas=3.0.5
numpy=2.4.6
joblib=1.6.0
fastapi=0.141.1
uvicorn=0.53.0
```

גרסאות נעוצות במפורש. קובץ בלי גרסאות בונה Image אחר בכל שבוע, וזה מה שגורם לתקלות שאי אפשר לשחזר.

DOCKERFILE

```

FROM python:3.12-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# שכבת התלויות נבנית לפני העתקת הקוד, כדי לנצל את משמון הבנייה
COPY requirements.txt .
RUN python -m pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY artifacts/ ./artifacts/

# הרצה כמשתמש לא root
USER 10001:10001

EXPOSE 8000

CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

.DOCKERIGNORE

```

.git/
.venv/
__pycache__/
.pytest_cache/
mlruns/
mlflow.db
test_*.py
train.py

```

BASH • בנייה והרצה

```

docker build -t iris-api:learning .
docker run --rm -p 127.0.0.1:8000:8000 iris-api:learning

```

עצור קודם שרת מקומי שמתמש באותו Port, אחרת הבקשה שלך תגיע לשירות הישן ותתבלבל מהתוצאה.

◆ ארבעה עקרונות בקובץ הקצר הזה

1. **סדר השכבות.** התלויות מותקנות לפני העתקת הקוד, כך ששינוי בקוד לא מפיל את מטמון ההתקנה. זה ההבדל בין בנייה של 5 שניות ל-3 דקות.
2. **משתמש לא-root.** אם מישהו יפרוץ לתהליך, ההרשאות שלו מוגבלות.
3. **הפרדת תלויות.** `pytest` ו-`train.py` אינם נכנסים ל-`Image` של הייצור.
4. **גרסה מפורשת.** אל תסתמך על `latest` לזיהוי גרסה – עקוב אחר תגית ו-`Digest`.

מדריכי Docker הרשמיים.

24 CI, CD ו-CT: שלושה תהליכים שונים

מטרת הפרק: לבנות שרשרת אוטומטית שעוצרת קוד שבור לפני שהוא מגיע למשתמשים.

קיצור	מה זה
CI – Continuous Integration	בדיקה אוטומטית של כל שינוי בקוד
Continuous Delivery	יצירת תוצר שעומד בתנאי שחרור וניתן לפריסה בלחיצה
Continuous Deployment	פריסה אוטומטית אחרי מעבר התנאים, בלי אישור ידני
CT – Continuous Training	תהליך חוזר של אימון על נתונים מתעדכנים

Pipeline בסיסי לפרויקט שלך

- 1 **קבלת שינוי בקוד**
– Push או Pull Request.
- 2 **התקנת התלויות שנבחרו**
– מהקובץ הנעוץ, לא מהאינטרנט החופשי.
- 3 **בדיקות קוד**
– lint, פורמט, בדיקת טיפוסים.
- 4 **בדיקות יחידה ואינטגרציה.**
- 5 **בדיקת תהליך האימון**
על מאגר קטן.
- 6 **שער איכות**
– האם המודל מנצח את ה-Baseline מספיק?
- 7 **בניית Image.**

– להרים את ה־Image ולשלוח לו בקשה אמיתית.

עם מזהה גרסה.

.GITHUB/WORKFLOWS/CI.YML

```
name: ml-service-ci

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: labs/iris-service

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Install dependencies
        run: python -m pip install -r requirements-dev.txt

      - name: Train a model
        run: python train.py

      - name: Run tests
        run: python -m pytest -q

      - name: Build the image
        run: docker build -t iris-api:${{ github.sha }} .

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: model-artifacts-${{ github.sha }}
          path: labs/iris-service/artifacts/
```

הקובץ המלא, כולל שער איכות ובדיקת עשן על הקונטיינר, נמצא ב־ `labs/iris-service/github-actions-ci.yml`.

לא כל Pipeline מאמן מודל מלא

במודל גדול, אין הכרח להריץ אימון מלא בכל Pull Request. אפשר לבדוק את הקוד ואת החוזים על מדגם קטן, ולהריץ אימון מלא בתהליך נפרד, מתוזמן או ידני. Pipeline שלוקח שעותיים הוא Pipeline שאנשים ילמדו לעקוף.

ההבדל החשוב ללמידה הוא לא שם הכלי, אלא איך מוגדרים השלבים, הכשלים, ההרשאות והתוצרים.

תרגיל 11

הכנס בכוונה שגיאה לחוזה הקלט – למשל שנה שם שדה ב- `app.py` בלי לעדכן את הבדיקות – ובדוק שה-Pipeline עוצר לפני יצירת תוצר לשחרור. אם הוא לא עוצר, חסרה לך בדיקה.

פריסה, Canary ו-Rollback

מטרת הפרק: לשחרר גרסה חדשה בלי להמר, ולדעת איך חוזרים אחורה כשצריך.

שיטה	איך היא עובדת	למה להשתמש בה
Rolling update	החלפה הדרגתית של מופעים	עדכון שירות בשלבים, בלי השבתה
Blue/Green	שתי סביבות מלאות והעברת תעבורה	מעבר וחזרה מהירים מאוד
Canary	חלק קטן מהתעבורה לגרסה החדשה	מידת השפעה לפני הרחבה
Shadow	הגרסה החדשה מקבלת עותק בקשות	בדיקה בייצור בלי להשתמש בתשובתה
A/B test	הקצאה אקראית לקבוצות	בחינת השפעה על מדד עסקי

Canary אינו ניסוי A/B

Canary בודק אם הגרסה החדשה *שבורה*. ניסוי A/B בודק אם היא *טובה יותר*, וזה דורש הקצאה אקראית, יחידת הקצאה מוגדרת, משך מוגדר מראש, גודל מדגם מספיק, ומחשבה על השפעה בין משתמשים. שימוש ב-Canary כדי להסיק מסקנות עסקיות הוא טעות נפוצה.

מה להגדיר לפני כל שחרור

- מה נחשב הצלחה, במספרים.
- מה מפעיל Rollback – ומי מוסמך להחליט.
- איזו גרסה היא נקודת החזרה.
- האם חוזה הנתונים תואם לשתי הגרסאות במקביל.
- מי מקבל החלטה במקרה לא חד-משמעי.

Rollback הוא לא רק קובץ המודל !

החזרה לגרסה קודמת חייבת לכלול את **כל** מה שנדרש להפעלתה: המודל, הקוד, עיבוד התכונות, ההגדרות, וסף ההחלטה. החלפת קובץ המודל בלבד, כשקוד העיבוד כבר התקדם, נותנת שילוב שמעולם לא נבדק. זו אחת התקלות הקשות ביותר לאבחון.

נתונים, תשתית ותפעול

מערכת שעובדת ביום הראשון היא הישג קטן. החלק הזה עוסק במה שנדרש כדי שהיא תמשיך לעבוד בחודש השישי, כשהנתונים משתנים והצוות גדל.

Data Engineering: להזרים מידע בצורה אמינה 26

מטרת הפרק: לבנות תהליכי נתונים שאפשר להריץ שוב בלי לשבור דברים.

מודל טוב צריך נתונים שמגיעים בזמן ובפורמט צפוי. שני מודלים של עיבוד:

- **Batch** מעבד קבוצת רשומות במועד מסוים – למשל כל לילה.
- **Streaming** מעבד אירועים שמגיעים ברצף לאורך הזמן.

תהליך לדוגמה: קריאת קבצים חדשים ← בדיקת Schema ← ניקוי ← יצירת תכונות ← כתיבת Dataset עם גרסה.

מושג	מה הוא מבטיח
Idempotency	הרצה חוזרת על אותו קלט אינה יוצרת תוצאות כפולות
Retries	ניסיון חוזר בכשל זמני, בלי התערבות אדם
Backfill	עיבוד תקופות היסטוריות אחרי תיקון או שינוי
Checkpoint	שמירת התקדמות, כדי להמשיך מאיפה שנעצרנו
Partitioning	חלוקת נתונים כדי לקרוא רק את הדרוש
Schema evolution	שינוי מבנה הנתונים תוך שמירת תאימות

Idempotency היא המושג החשוב כאן

תהליך שרץ פעמיים בטעות – וזה יקרה – חייב להשאיר את המערכת באותו מצב. בפועל זה אומר: כתיבה לפי מפתח במקום הוספה, מחיקת מחיצה לפני כתיבה מחדש, או `UPSERT` במקום `INSERT`. תהליך שאינו כזה יוצר נתונים כפולים שמזהמים את מאגר האימון חודשים אחורה.

Airflow מתזמן ומנהל תלויות בין משימות **Spark** מבצע עיבוד נתונים מבוזר. **Kafka** מספק תשתית לזרמי אירועים. אלה תפקידים שונים לחלוטין, גם אם המוצרים משתלבים באותה מערכת.

תרגיל 12

כתוב תהליך יומי שמפיק תחזיות לקובץ. הרץ אותו פעמיים על אותו יום, והוכח בקוד שלא נוצרו כפילויות. אחר כך הרץ Backfill על שבוע אחורה.

27 אחסון, גרסאות נתונים ו־Feature Stores

מטרת הפרק: לדעת איפה כל סוג מידע צריך לשבת, ולמה תכונות דורשות טיפול מיוחד.

מידע	מקום אפשרי
קבצים גולמיים ומודלים גדולים	אחסון אובייקטים, כגון S3
מידע עסקי עם קשרים וטרנזקציות	מסד רלציוני
Metadata של ניסויים	מסד הנתונים של מערכת המעקב
תכונות שנדרשות בחיזוי מהיר	Online store מתאים
לוגים לחיפוש וחקירה	מערכת חיפוש ולוגים
תמונות Docker	Container registry

גרסאות נתונים

קוד מנוהל ב־Git. נתונים לא – הם גדולים מדי. אבל בלי גרסה לנתונים, אי אפשר לשחזר ניסוי. שלוש גישות מקובלות:

- **Immutable snapshots**: כל מאגר אימון נכתב לנתיב עם תאריך וגרסה, ולא נדרס לעולם.
- **Hash של התוכן**: שמירת `sha256` של המאגר לצד הניסוי – כמו שעשינו ב־`train.py`.
- **כלי ייעודי**: **DVC**, **LakeFS**, או פורמטים כגון **Delta Lake** ו־**Apache Iceberg** שתומכים ב"נסיעה בזמן".

Feature Store

Feature Store מסייע בניהול תכונות משותפות ובאספקתן גם לאימון וגם לחיזוי. הוא **אינו חובה** לכל פרויקט, והוספתו מוקדם מדי היא סיבוך מיותר.

ההבחנה החשובה היא בין **Offline features** – יצירת נתוני אימון היסטוריים, לבין **Online features** – שליפת ערכים עבור בקשה שמגיעה כעת.

Point-in-time correctness !

באימון, צריך לחבר לכל דוגמה את ערכי התכונות שהיו זמינים באותו רגע בהיסטוריה. חיבור לערכים המעודכנים ביותר – מה ש־`JOIN` רגיל יעשה – יוצר דליפה מהעתיד, והמודל ייראה מצוין באימון ויתרסק בייצור. זו הבעיה המרכזית ש־Feature Store נועד לפתור (**הסבר Feast**).

Elasticsearch שימושי לחיפוש טקסט ולניתוח לוגים. הכר Document, Index, Mapping, Shard ו-Replica, ואת ההבחנה בין שדה טקסט שעובר ניתוח לבין ערך `keyword` מדויק (יסודות האנדקסים).

Kubernetes: להבין את הבעיה לפני הכלי 28

מטרת הפרק: להכיר את מושגי היסוד, ולדעת מתי בכלל להגיע לשם.

כאשר מפעילים שירותים רבים, או כשנדרשת פלטפורמה משותפת לכמה צוותים, צריך לנהל מופעים, משאבים, פריסות והתאוששות. זו הבעיה ש-Kubernetes פותר.

מושג	מה זה
Cluster	קבוצת משאבים שמנוהלת יחד
Node	מכונה שמשתתפת באשכול
Pod	יחידת ההרצה הבסיסית
Deployment	ניהול מופעים ועדכונים
Service	כתובת יציבה לגישה למופעים
Requests / Limits	בקשות משאבים ומגבלות
HPA	התאמת מספר מופעים לפי מדדים

שלוש בדיקות בריאות, שלוש מטרות

Probe	מטרה	מה קורה בכישלון
Startup	לתת לתהליך להשלים אתחול	ה-Pod מופעל מחדש
Readiness	לקבוע אם אפשר לשלוח תעבורה	התעבורה מופסקת, ה-Pod ממשיך לרוץ
Liveness	לזהות מצב שמצדיק אתחול מחדש	ה-Pod נהרג ומופעל מחדש

! התקלה הקלאסית של שירותי ML

שירות שטוען מודל גדול עשוי להזדקק לדקה או שתיים של אתחול. בדיקת Liveness אגרסיבית מדי תהרוג אותו לפני שסיים לעלות, והוא ייכנס ללולאת אתחול אינסופית שנראית כמו תקלת זיכרון. זו בדיוק המטרה של Startup probe.

```

startupProbe:
  httpGet: { path: /health, port: 8000 }
  failureThreshold: 30      # עד 5 דקות לעליית המודל
  periodSeconds: 10

readinessProbe:
  httpGet: { path: /ready, port: 8000 }
  periodSeconds: 5

livenessProbe:
  httpGet: { path: /health, port: 8000 }
  periodSeconds: 20

resources:
  requests: { cpu: "500m", memory: "16i" }
  limits:   { cpu: "2",    memory: "26i" }

```

מתי להגיע ל-Kubernetes

בשלב הלימוד הראשון, שירות מקומי וקונטיינר מספיקים לחלוטין. עבור לתרגול Kubernetes רק אחרי שאתה יודע להפעיל, לחקור ולנפות קונטיינר בודד. אחרת אתה מנפה שתי שכבות שאינך מכיר בו־זמנית.

מושגי Kubernetes, תיעוד Probes.

29 ניטור: למדוד שירות, נתונים ומודל

מטרת הפרק: לדעת מה למדוד בכל שכבה, ולהבין למה שירות בריא יכול להחזיר תחזיות גרועות.

מערכת יכולה להחזיר תשובות במהירות, בלי שגיאה אחת, ועדיין להיות שבורה לגמרי. זה ההבדל בין ניטור תוכנה לניטור ML

שכבה	מה מודדים
שירות	זמני תגובה, שיעור שגיאות, תעבורה ומשאבים
נתונים	חוסרים, סוגים, טווחים, קטגוריות חדשות וטריות המידע
תחזיות	התפלגות הציונים, יחס בין מחלקות, ומעבר סף
איכות מודל	מדדים אמיתיים, כאשר מגיעות תשובות אמת
עסק	השפעה על התהליך שהמערכת נועדה לשפר

ארבעה מושגים שחייבים להבדיל ביניהם

CONCEPT DRIFT

הקשר השתנה

אותו קלט מוביל עכשיו לתשובה אחרת. זה מה שבאמת שובר מודלים.

DATA DRIFT

הקלט השתנה

התפלגות התכונות זזה. למשל, לקוחות מסוג חדש. לא בהכרח אומר שהמודל נפגע.

DELAYED LABELS

התשובה מאחרת

התוצאה האמיתית מגיעה רק בעוד שבועיים. עד אז אתה עיוור לאיכות.

TRAINING-SERVING SKEW

פער בין עולמות

העיבוד באימון והעיבוד בשירות אינם זהים. באג, לא תופעה.

שינוי בקלט אינו הוכחה להידרדרות

התראת drift אינה עילה אוטומטית לאימון מחדש. מנגד, האיכות יכולה להיפגע גם בלי שינוי גדול שקל לזהות בקלט. drift הוא סימן לחקירה, לא מסקנה.

כדי למדוד איכות בפועל, שמור מזהה שמאפשר לחבר תחזית לתוצאה שקרתה – בדיוק ה- `prediction_id` שהוספנו בפרק 19 – יחד עם גרסת המודל וזמן החיזוי. קבע מדיניות שמירה מתאימה למידע שאתה מתעד.

SLI ו-SLO

SLI הוא המדד עצמו. **SLO** הוא היעד שקבעת לו. לדוגמה: "99% מהבקשות התקינות מסתיימות בתוך 200 מילישניות, במדידה על פני 30 יום". זהו יעד תרגול, לא המלצה גורפת. ראה ספר ה-SRE של Google.

טיפול בתקלות הוא מיומנות מרכזית 30

מטרת הפרק: לאבחן לפי ראיות במקום לנחש, ולצאת מכל תקלה עם בדיקה חדשה.

תרחיש א': זמן התגובה עלה אחרי פריסה

1 זהה מתי התחילה העלייה

– לפי הגרפים, לא לפי תחושה.

2 בדוק אילו שינויים התרחשו בסמוך

– קוד, מודל, תלות, הגדרה, או תעבורה.

3 הפרד את הזמן לרכיבים

– רשת, שליפת תכונות, עיבוד, חיזוי.

4 בדוק עומס, תורים, זיכרון ושגיאות.

5 השווה בין הגרסה הישנה לחדשה

על אותה בקשה.

6 בצע Rollback

אם תנאי ההחזרה מתקיימים. קודם מייצבים, אחר כך חוקרים.

7 תקן, והוסף בדיקה

שהייתה תופסת את זה מראש.

תרחיש ב': איכות המודל ירדה

1 ודא שמדידת האיכות עצמה נכונה.

לעיתים קרובות המדידה נשברה, לא המודל.

2 בדוק שה-Labels הגיעו

והתחברו לתחזיות הנכונות.

3 בדוק שינוי Schema, יחידות או סדר תכונות.

4 בדוק איזה פלח נפגע

— לרוב לא כולם.

5 בדוק שינוי באוכלוסייה או במוצר

— אולי הושקה תכונה חדשה.

6 רק אז החלט

אם אימון נוסף הוא בכלל הפתרון.

תרגיל 13 • שבעה תרגילי תקלה על הפרויקט שלך

1. החלף בין שתי תכונות ברשימת **FEATURES**.

2. שנה את היחידות מסנטימטרים למילימטרים בקלט בלבד.

3. טען מודל שאומן עם גרסת ספרייה אחרת.

4. שלח בקשה עם שדה חסר.

5. הגבל את זיכרון הקונטיינר ל-128MB.

6. הפסק שירות שהמערכת תלויה בו.

7. הפעל שוב תהליך Batch שכבר הסתיים.

לכל תקלה כתוב חמש שורות: **סימפטום, איך זוהה, גורם שורש, התאוששות, מניעה**. התייעוד הזה שווה בראיון יותר מכל פרויקט נוסף.

מטרת הפרק: להגן על המערכת, על הנתונים ועל המשתמשים – ולדעת להוכיח מה רץ ולמה.

מערכת ML כוללת קוד, מידע ומודלים, וכל אחד מהם דורש ניהול מתאים.

עקרונות עבודה

- השתמש בהרשאות מצומצמות לכל רכיב. שירות החיזוי לא צריך גישה כתיבה למאגר האימון.
- שמור סודות מחוץ לקוד ומחוץ ל־Images. השתמש במנגנון ניהול סודות.
- בדוק קלט וגודל בקשות – גם כהגנה מפני עומס.
- קבע Authentication ו־Authorization כאשר השירות נגיש למשתמשים.
- תעד מי אישר ומי פרס כל גרסה.
- טען מודלים רק ממקורות מהימנים (ראה אזהרת `pickle` בפרק 18).
- בדוק תלויות ורישיונות לפני שימוש במוצר.
- צמצם מידע אישי בלוגים ובדוגמאות ניסוי.

פרטיות

נושא	מה לעשות
מזעור נתונים	אל תאסוף שדה שאין לו שימוש מוגדר
מדיניות שמירה	קבע מראש כמה זמן נשמרים לוגים ותחזיות
זכות למחיקה	ודא שאתה יכול למחוק נתוני אדם – גם ממאגרי אימון
הסתרה בלוגים	אל תרשום מזהים אישיים בלוג התחזיות

הזרקת הוראות במערכות LLM

⚠ טקסט שנקרא אינו הוראה מורשית

ב־LLM עם כלים, יש הבדל מהותי בין טקסט שהמודל *קורא* לבין הוראות מורשות למערכת. מסמך שנשלף מאינדקס אינו מקבל הרשאה לשנות מדיניות, לחשוף סודות, או להפעיל פעולה. אם הארכיטקטורה שלך לא מבחינה בין השניים – יש לך פרצה, לא תכונה.

ממשל: Model Governance

בארגונים גדולים ובתחומים מפוקחים, נדרש תיעוד מסודר של כל מודל בייצור: מה הוא עושה, על מה אומן, מה מגבלותיו, מי אישר אותו, ומתי נבדק לאחרונה. המסמך הזה נקרא לעיתים **Model Card**.

האם אני יכול להוכיח מהיכן הגיע המודל שרץ עכשיו, על אילו נתונים אומן, באילו תנאים נבדק, ומי אישר אותו?

אם התשובה היא לא – יש לך בעיית ממשל, גם אם המערכת עובדת מצוין היום. זו גם שאלה שנשאלת כמעט תמיד בראיון לתפקיד MLOps בכיר.

הוגנות

בדוק מדדים לפי קבוצות רלוונטיות, לא רק ממוצע כללי. מודל שמדויק ב-92% בממוצע יכול להיות מדויק ב-60% בפלח מסוים, ואם ההחלטה משפיעה על אנשים – זו לא הערת שוליים טכנית.

מתקדם

למידה עמוקה, מודלי שפה, ביצועים וענן. הגיע לכאן רק אחרי שהפרויקט מחלק ג' עובד אצלך מקצה לקצה.

Deep Learning: איך רשת מתאמנת 32

מטרת הפרק: להבין את מכניקת האימון, לא לשנן ארכיטקטורות.

רשת עצבית משלבת שכבות של פעולות נלמדות ופונקציות לא ליניאריות.

מושג	משמעות
Tensor	מערך רב־ממדי
Layer	טרנספורמציה בקלט
Activation	פונקציה שמאפשרת התנהגות לא ליניארית
Forward pass	חישוב התחזית
Loss	מדידת הטעות
Backpropagation	חישוב גרדיאנטים לאחור
Optimizer	עדכון הפרמטרים
Batch	קבוצת דוגמאות לעדכון אחד
Epoch	מעבר שלם על נתוני האימון

```

import torch
from torch import nn

torch.manual_seed(11)

X = torch.randn(256, 3)
y = (2 * X[:, 0] - 3 * X[:, 1] + 0.5 * X[:, 2]).unsqueeze(1)

model = nn.Sequential(
    nn.Linear(3, 16),
    nn.ReLU(),
    nn.Linear(16, 1),
)

loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

model.train()

for step in range(200):
    optimizer.zero_grad()           # מאפסים גרדיאנטים מהצעד הקודם
    predictions = model(X)          # forward
    loss = loss_fn(predictions, y)
    loss.backward()                 # backward
    optimizer.step()                # עדכון פרמטרים

model.eval()

with torch.inference_mode():
    example_prediction = model(X[:1])

print(example_prediction)

```

שתי הערות על הקוד

הדוגמה מסבירה את **מכניקת האימון** על מידע מלאכותי. כדי לבדוק הכללה צריך להוסיף נתונים נפרדים להערכה – כאן אין כאלה בכוונה.

משנה את התנהגותן של שכבות מסוימות, כגון Dropout ו-BatchNorm. הוא **אינו** תחליף לכיבוי חישוב גרדיאנטים; `inference_mode()` מטפל בהיבט אחר לגמרי. `model.eval()`

בהמשך למד DataLoader, שמירת Checkpoints, CNN, Transfer learning, Attention ו-Transformers. מקורות: **יסודות PyTorch**, תהליך האופטימיזציה.

מתי בכלל להשתמש ברשת

לנתונים טבלאיים, Gradient Boosting לרוב עדיף – מהיר יותר, זול יותר, וקל יותר להסביר. רשתות מנצחות בבירור בתמונות, אודיו, טקסט ורצפים. אל תבחר רשת כי היא מרשימה; בחר אותה כשסוג הנתונים מצדיק זאת.

RAG, NLP, Embeddings

33

מטרת הפרק: לעבד טקסט, להבין ייצוגים, ולבנות מערכת שליפה שאפשר להעריך.

טקסט צריך לעבור לייצוג שהמודל יכול לעבד. בשלב הראשון כדאי ללמוד בסדר הזה:

1 Tokenization

– חלוקה ליחידות טקסט.

2 Bag of Words ו-TF-IDF

– ייצוגים פשוטים שעדיין מנצחים הרבה בעיות.

3 סיווג טקסט

עם מודל פשוט. זה ה-Baseline שלך.

4 Embeddings

– ייצוג מספרי נלמד שתופס משמעות.

5 Similarity

– מדידת דמיון בין ייצוגים.

LLM מעבד רצפי Tokens. ביצירת טקסט הוא מפיק התפלגות עבור ההמשך, לפי המודל וההקשר שקיבל.

גישה	מה משתנה	מתי מתאים
Prompting	ההוראות והקלט	נקודת הפתיחה תמיד
RAG	המידע שנשלף ומתווסף להקשר	ידע שמשתנה, או מידע ארגוני
Fine-tuning	פרמטרים נלמדים של המודל	סגנון, פורמט, או משימה צרה וחוזרת
שימוש בכלים	היכולת לבצע פעולות או לשאול מערכות	כשצריך מידע חי או פעולה אמיתית

RAG: מה בעצם קורה

בזרימה טיפוסית: מחלקים מסמכים לקטעים, יוצרים אינדקס, שולפים קטעים מתאימים לשאלה, ומספקים אותם למודל כהקשר.

שש בדיקות נפרדות למערכת RAG

1. האם השליפה מצאה את המידע הנכון? (מדד שליפה, בנפרד לגמרי מהתשובה)
2. האם התשובה נאמנה למידע שנשלף, או שהמודל הוסיף מעצמו?
3. האם המקורות שצוטטו באמת תומכים בטענות?
4. האם הרשאות המסמכים נשמרות? משתמש לא אמור לקבל קטע ממסמך שאין לו גישה אליו.
5. האם זמן התגובה והעלות מתאימים לשימוש?
6. מה קורה כשהמידע פשוט לא נמצא? "אינני יודע" היא תשובה תקינה שצריך לתכנן.

שתי אמונות שגויות נפוצות

RAG אינו מבטיח תשובה נכונה – הוא רק מספק הקשר טוב יותר. אם השליפה החזירה את המסמך הלא נכון, התשובה תהיה שגויה בביטחון מלא.

Fine-tuning אינו תחליף למאגר מידע שמתעדכן. הוא משנה התנהגות וסגנון, לא מוסיף עובדות טריות באופן אמין.

למסלול המשך: קורס ה-LLM של Hugging Face.

LLMOps: להפעיל מערכות LLM בייצור 34

מטרת הפרק: להבין מה שונה בתפעול מערכת מבוססת מודל שפה, לעומת מודל טבלאי.

הרבה ממה שלמדנו בחלקים ג' וד' נשאר תקף: גרסאות, בדיקות, ניטור, Rollback. אבל ארבעה דברים משתנים מהותית.

2

ה-Prompt הוא קוד

הוא חייב לשבת ב-Git, לעבור גרסה, ולהיבדק. שינוי ניסוח שנעשה ידנית בייצור הוא פריסה לא מתועדת.

1

הפלט אינו דטרמיניסטי

אותו קלט יכול לתת פלט שונה. בדיקה של "האם הפלט שווה למחרוזת הצפויה" לא תעבוד. צריך בדיקות תכונה: אורך, פורמט, נוכחות שדות, ואיחופעה של מידע רגיש.

4

ההערכה עצמה קשה

אין `f1_score` אחד. צריך מערך בדיקה ידני, ולעיתים מודל שופט – שגם אותו צריך לאמת מול שיפוט אנושי.

3

העלות משתנה לכל בקשה

בניגוד למודל טבלאי, העלות תלויה באורך הקלט והפלט. חובה למדוד tokens לכל בקשה ולתקצב.

מה למדוד בייצור

מדד	למה
Tokens לכל בקשה	הבסיס לתקציב ולזיהוי בקשות חריגות
Latency, כולל time-to-first-token	בממשק זורם, ההתחלה חשובה יותר מהסוף
שיעור תשובות ריקות או חסומות	סימן לבעיה בהוראות או במסנן
שיעור שימוש בכלים ושיעור כישלון שלהם	כלי שנכשל שקט הוא מקור לתשובות שגויות
משוב משתמשים	לעיתים המדד היחיד שבאמת קיים
מדדי שליפה (ב-RAG)	נמדדים בנפרד מאיכות התשובה

מעריך בדיקה הוא הנכס החשוב ביותר

לפני שמכווננים prompt, בנה קובץ של 50–200 מקרים אמיתיים עם התשובה הרצויה או עם קריטריון קבלה. בלעדיו כל שינוי הוא תחושה. עם קובץ כזה, אתה יכול להשוות גרסאות, מודלים וספקים – וזה בדיוק מה שמבדיל מערכת מהונדסת מאוסף ניסויים.

תרגיל 14

קח משימה אחת של סיווג טקסט. בנה 50 מקרי בדיקה עם תשובה נכונה. הרץ עליהם שתי גרסאות prompt, ומדוד הבדל. עכשיו יש לך תהליך הערכה, לא הרשמה.

מטרת הפרק: לאתר את צוואר הבקבוק האמיתי במקום לזרוק חומרה על הבעיה.

מודל מורכב יותר אינו בהכרח הפתרון המשתלם, ומכונה גדולה יותר אינה בהכרח הפתרון המהיר.

מדד	מה הוא אומר
Latency	זמן תגובה לבקשה בודדת. מדוד p50, p95, ו-p99, לא ממוצע.
Throughput	כמות בקשות ליחידת זמן
Concurrency	מספר פעולות שמתבצעות במקביל
זיכרון ו-CPU/GPU	מה באמת נצרך, ומה מוגדר כמגבלה
זמן טעינה	כמה זמן לוקח למופע חדש להיות מוכן
עלות לחיזוי	המספר שמעניין את מי שמאשר תקציב

חוק Little

בתנאים מתאימים, ממוצע מספר הבקשות במערכת שווה לקצב ההגעה כפול זמן השהייה הממוצע:

$$\text{בקשות/שנייה} \times 0.05 \text{ שניות} = 200 \times 10$$

זה אינו מספר ה־Workers המומלץ. צריך להביא בחשבון שונות, תורים, מקביליות וסוג העבודה. זו נקודת פתיחה לחישוב, לא תשובה.

דוגמת חישוב אחסון

מיליון Embeddings בגודל 768, עם ארבעה Bytes לכל רכיב:

$$1,000,000 \times 768 \times 4 \text{ bytes} \approx 2.86 \text{ GiB}$$

זה רק הערכים עצמם. אינדקס החיפוש, Metadata ועותקים מוסיפים נפח משמעותי מעבר לכך.

לפני שמוסיפים GPU או מכונות

בדוק היכן הזמן באמת נשרף. לעיתים קרובות צוואר הבקבוק הוא קריאת נתונים מהדיסק, המרה ב־pandas, המתנה לשירות חיצוני, או סריאליזציה של JSON – ולא החיזוי עצמו. מדידה של שעה חוסכת חשבון ענן של חודש.

מטרת הפרק: להעביר את המערכת המקומית לענן תוך שמירה על אותם עקרונות.

אחרי שהמערכת המקומית עובדת מקצה לקצה, העבר אותה לענן. המיפוי הבא הוא נקודת התחלה, לא מרשם:

צורך	רכיב AWS אפשרי
נתונים ומודלים	S3
Images	ECR
שירות בקונטיינר	ECS/Fargate או פתרון מתאים אחר
עבודה מנוהלת עם מודלים	SageMaker AI
ניהול זהויות	IAM Roles
סודות	Secrets Manager
לוגים ומדדים	CloudWatch
תור לעבודות	SQS
תזמור שלבים	Step Functions או כלי תזמור אחר

SageMaker AI מספק יכולות לבנייה, אימון ופריסה. **Model Monitor** מספק יכולות ניטור מוגדרות – אבל עדיין צריך לתכנן מה מודדים, מתי מתקבלים Labels, ומה עושים בעקבות התראה. כלי לא מחליף חשיבה.

!

לפני שאתה פותח משאב בענן

הגדר מראש אילו רכיבים נוצרים, איך נמדדת העלות, ואיך מסירים אותם בסיום. **אל תניח שרכיב שאינו מקבל בקשות אינו מחויב**. אשכולות, כתובות שמורות, נפחי אחסון ונקודות קצה מנוהלות ממשיכים לעלות כסף גם בשקט מוחלט.

▶

תרגיל 15 • פרויקט ענן ראשון

העלה את ה־Image ל־Registry, הפעל שירות עם Role מצומצם, שמור מודל עם גרסה, הוסף ניטור בסיסי, ובצע החזרה לגרסה קודמת. בסוף – הסר הכול, ובדוק בחשבונית שאכן הוסר.

מהמדריך לעבודה

ידע שלא ניתן להראות אינו נחשב בשוק העבודה. החלק הזה עוסק בהפיכת מה שלמדת לתיק עבודות, לתוכנית מסודרת, ולתשובות שאתה יכול להגן עליהן.

לבנות תיק עבודות שמוכיח יכולת 37

מטרת הפרק: לבנות שלושה פרויקטים שמוכיחים שאתה מהנדס, לא שעברת קורס.

אחרי מעבדת Iris, בנה שלושה פרויקטים משמעותיים. שלושה שאתה מבין לעומק שווים הרבה יותר מעשרים מחברות שהרצת.

פרויקט	מה בונים	מה הוא מוכיח
1. חיזוי על נתונים טבלאיים	זמן, ביקוש, נטישה או סיכון	הגדרת בעיה, נתונים, מודלים והערכה
2. מערכת MLOps	אימון, CI, API, Registry ו-Rollback	הנדסה, אוטומציה ותפעול
3. מערכת טקסט	סיווג פניות או חיפוש מבוסס מסמכים	NLP, הערכה ותכנון מוצר

פרויקט 1 – חיזוי טבלאי

השתמש בנתונים ציבוריים מתאימים או בנתונים שאתה רשאי להשתמש בהם. הגדר רגע חיזוי, Baseline וחלוקה נכונה. השווה מספר מועמדים ונתח טעויות לפי פרק 17. אם אתה משתמש במידע מלאכותי – ציין זאת במפורש, ואל תסיק ממנו ביצועים בעולם האמיתי.

פרויקט 2 – מערכת MLOps

קח את המודל מפרויקט 1 והפוך אותו למערכת: גרסאות, API, Docker, תהליך בדיקות, רישום ניסויים, מדדים, ותרגיל התאוששות מתועד. זה הפרויקט שמרשים בראיון ל-MLOps.

פרויקט 3 – מערכת טקסט

בנה מערכת טקסט עם קבוצת בדיקה שאתה יכול להסביר. למשל, סיווג פניות והשוואה בין TF-IDF עם מודל פשוט לבין ייצוגים נלמדים. אם בחרת RAG – בדוק גם שליפה וגם תשובה, בנפרד.

מה כל פרויקט חייב לכלול

- ☐ README עם הוראות הפעלה שעובדות על מחשב נקי
- ☐ הגדרת הבעיה ורגע החיזוי
- ☐ מקור הנתונים והמגבלות שלהם
- ☐ תיאור החלוקה, ולמה דווקא היא
- ☐ תוצאות אמיתיות, כולל ה-Baseline
- ☐ דוגמאות לכשלים של המודל
- ☐ החלטות אדריכליות ונימוקים
- ☐ בדיקות שרצות
- ☐ הוראות פריסה והחזרה
- ☐ מה היית משפר בשלב הבא

שתי טעויות שהורגות תיק עבודות

מחברת בלי הקשר. Notebook עם accuracy 0.97 ובלי הסבר מה הבעיה ומה ה-Baseline אינו אומר דבר על יכולתך.

README שלא עובד. אם המראיין מנסה להריץ ונכשל בשלב השני, הפרויקט נגמר. בדוק את ההוראות שלך על סביבה נקייה.

תוכנית עבודה מוצעת 38

מטרת הפרק: להמיר את המדריך ללוח זמנים מציאותי, עם תוצר מדיד בכל שלב.

מסלול מאפס דורש זמן משמעותי. הטבלה הבאה היא מסגרת תכנון, לא הבטחה לקצב או לקבלה לעבודה.

שלב	פרקים	היקף מוצע	תוצר מעבר
Python וחשיבה תכנותית	5-2	80-120 שעות	תוכנית קטנה עם קלט, קבצים ובדיקות
Linux, Git, SQL ונתונים	8-6	70-100 שעות	תהליך שמפיק טבלה תקינה ממקורות
מתמטיקה ו-ML בסיסי	16-9	100-140 שעות	Baseline, מודל והערכה ללא דליפה
פרויקט ML ראשון	18-17	60-90 שעות	דוח ניסויים וניתוח טעויות
API ו-MLOps, Docker	25-19	100-140 שעות	שירות עם בדיקות ותהליך שחרור
תשתית, ניטור ותפעול	31-26	70-100 שעות	תהליך נתונים, ניטור ותרגיל תקלה
Deep Learning ו-NLP	34-32	80-120 שעות	פרויקט נוסף עם הערכה
ענן, ביצועים וראיונות	40-35	70-100 שעות	פריסה, תרגיל תקלה והצגת פרויקט

סך הכול כ-630-910 שעות. בקצב של 12 שעות בשבוע, זה בערך 14-18 חודשים. בקצב של 25 שעות בשבוע – כ-7-9 חודשים. התאם את הקצב לפי היכולת לבצע, לא לפי מספר הפרקים שקראת.

חלוקה שבועית אפשרית

1 שני מפגשים להבנת חומר
– קריאה, וידאו, תיעוד רשמי.

2 שני מפגשים לקוד ותרגול
– לבד, בלי להעתיק פתרונות.

3 מפגש אחד לפרויקט
– התקדמות רציפה בפרויקט הנוכחי.

4 חזרה קצרה
על טעויות השבוע, והסבר בעל פה של מה שלמדת.

מבחן סיום לכל יחידה

1. סגור את החומר.
2. הסבר את הרעיון במילים שלך, בקול.
3. בצע משימה דומה בלי להעתיק.
4. שנה דרישה אחת במשימה.
5. ודא שהפתרון עדיין עובד – ושאתה מבין למה.

אפשר ורצוי להשתמש בעוזר AI לרמזים, להסברים ולביקורת קוד. הכלל היחיד: ודא שאתה מסוגל להסביר את הפתרון ולתקן בו תקלה בעצמך. אם אינך יכול – קיבלת קוד, לא יכולת, ובראיון זה יתגלה תוך חמש דקות.

שאלות שאתה צריך לדעת לענות עליהן 39

מטרת הפרק: להתכונן לשיחה מקצועית – לא בשינון, אלא בקישור לפרויקטים שלך.

שאלה	מה צריכה לכלול תשובה טובה
מה ההבדל בין Training ל-Inference?	למידת פרמטרים לעומת שימוש בהם
למה צריך Baseline?	לבדוק אם המורכבות מוסיפה ערך
מהי Data leakage?	מידע שלא היה זמין במצב החיזוי האמיתי
איך מחלקים נתוני זמן?	אימון על עבר, הערכה על תקופות מאוחרות
למה Accuracy יכולה להטעות?	חוסר איזון ועלויות שונות של טעויות
מה ההבדל בין Precision ל-Recall?	אמינות ההתרעות לעומת שיעור המקרים שזוהו
מה עושה Regularization?	מגבילה את פתרון הלמידה כדי לשפר הכללה
למה לשמור Pipeline ולא רק מודל?	לשמר את עיבוד הנתונים יחד עם המודל
מה צריך לשחזור ניסוי?	קוד, נתונים, סביבה, חלוקה והגדרות
מה ההבדל בין Artifact ל-Registry?	תוצר שנשמר לעומת ניהול זהות וגרסאות
מתי לבחור Batch inference?	כאשר התוצאה אינה נדרשת מיד לכל בקשה
למה המודל איטי בייצור?	פירוק המדידה: נתונים, הכנה, חיזוי, תורים ורשת
מהי Readiness, ומה ההבדל מ-Liveness?	האם לשלוח תעבורה, לעומת האם להפעיל מחדש
האם Drift מחייב Retraining?	לא; קודם בודקים מקור, השפעה ופתרון
מהו Training-serving skew?	פער בקלט או בעיבוד בין האימון לשירות
איך בודקים מודל לפני פריסה?	איכות, קבוצות, חוזה, ביצועים ואמינות
איך מבצעים Rollback?	שחזור חבילת הגרסה המלאה, לא רק קובץ המודל
מה עושה Feature Store?	ניהול ואספקת תכונות, כולל נכונות נקודת-זמן
מה ההבדל בין RAG ל-Fine-tuning?	שינוי ההקשר שנשלף לעומת שינוי פרמטרים
מה עשית כשהמערכת נכשלה?	ראיות, אבחון, התאוששות ושיפור מנגנוני הבדיקה

"הבעיה הייתה... הגדרתי הצלחה באמצעות... הנתונים הגיעו מ... מנעתי דליפה באמצעות... השוויתי ל... בחרתי בפתרון הזה משום ש... פרסתי אותו כך... הכשל המרכזי שמצאתי היה... ושיניתי..."

אל תשנן את הניסוח. קשר כל תשובה לפרויקט שביצעת בפועל – זה ההבדל בין מועמד שקרא לבין מועמד שבנה.

שאלות שכדאי שתשאל אתה

– איך נראה תהליך שחרור מודל אצלכם?

התשובה מגלה מיד את רמת הבשלות של הצוות, וכמה עבודת תשתית תחכה לך.

– איך אתם יודעים שמודל בייצור עדיין עובד טוב?

אם אין תשובה ברורה, זו הזדמנות עבורך – או סימן אזהרה, תלוי איך זה מוצג.

– מי מחליט אם מודל נכנס לייצור?

מגלה את חלוקת האחריות בין מחקר, הנדסה ומוצר.

– מה הייתה התקלה האחרונה במערכת ML אצלכם, ואיך טופלה?

שאלה מצוינת. היא גם מראה שאתה חושב כמו מי שמתפעל מערכות, לא רק בונה אותן.

40 איך תדע שאתה מוכן

מטרת הפרק: מבחן מעשי לעצמך, לפני שאתה מתחיל להגיש מועמדות.

בדוק יכולת מעשית במשימה שלא פתרת קודם. סמן רק מה שאתה יכול לעשות עכשיו, בלי עזרה:

מבחן מוכנות

- ☐ לקרוא נתונים ולזהות בהם בעיות משמעותיות
- ☐ לכתוב Python ו-SQL בלי להסתמך על העתקה
- ☐ להגדיר יעד וחלוקת נתונים מתאימים לשימוש האמיתי
- ☐ לבנות Baseline ומודל, ולהשוות ביניהם נכון
- ☐ להסביר את בחירת המדדים לאדם לא טכני
- ☐ לזהות דליפת מידע בתיאור בעיה
- ☐ לנהל ניסויים וגרסאות כך שאפשר לשחזר אותם
- ☐ לשמור ולטעון מודל באופן עקבי
- ☐ להפעיל שירות חיזוי עם חוזה מוגדר
- ☐ לכתוב בדיקות משמעותיות לקוד, לנתונים ולמודל
- ☐ לארוז ולהריץ בקונטיינר
- ☐ לאבחן תקלה באמצעות לוגים ומדדים
- ☐ להסביר החלטות ופשרות שעשית, ולהגן עליהן
- ☐ להראות פרויקטים שניתנים לשחזור על מחשב אחר

העמקה לפי מסלול

MLOPS ENGINEER

איפה להעמיק

Linux, רשתות, CI/CD, ענן, תזמור, הרשאות ותפעול.
פרקים 22–31 הם הליבה שלך.

ML ENGINEER

איפה להעמיק

הכנת נתונים, הערכה, מודלים, ניתוח טעויות, ותכנון
תכונות. פרקים 12–18 הם הליבה שלך.

ומה אחרי

המשך הדרך למומחיות אינו עוד קורס. הוא עבודה על מערכות לאורך זמן: נתונים שמשתנים, דרישות חדשות, מגבלות תקציב, תקלות בשעות לא נוחות, שיתוף פעולה עם אנשים שחושבים אחרת, ואחריות על החלטות שיש להן השלכות.

המשימה הראשונה שלך, היום

הקם סביבת Python, כתוב חמש פונקציות קטנות, שמור אותן ב-Git, ובנה תוכנית שקוראת קובץ נתונים ומפיקה סיכום.

לפני שאתה עובר למודל הראשון, ודא שאתה מבין כל שורה שכתבת, ושאתה יכול לשנות את התוכנית בעצמך.

חומר עזר

מילון מונחים, צ'קליסט שחרור לייצור, מקורות מומלצים, ורשימת הטעויות שחוזרות אצל כולם.

המונחים שתפגוש בכל תיאור משרה, בכל ראיון ובכל שיחת צוות.

מונח	פירוש
A/B test	ניסוי מבוקר שמשווה שתי גרסאות על מדד עסקי, עם הקצאה אקראית
Artifact	תוצר שנשמר מתהליך: מודל, מדדים, מטא-דאטה
Backfill	הרצת תהליך על תקופות היסטוריות
Baseline	הפתרון הפשוט ביותר שכל מודל חייב לנצח
Batch inference	הפקת תחזיות לקבוצת רשומות במועד מתוזמן
Calibration	האם הסתברויות המודל משקפות תדירות אמיתית
Canary	פריסה לחלק קטן מהתעבורה לפני הרחבה
Checkpoint	שמירת מצב ביניים של אימון או תהליך
CI / CD / CT	אינטגרציה, אספקה ואימון רציפים
Concept drift	הקשר בין הקלט לתשובה השתנה
Container	מופע רץ של Image
Cross-validation	הערכה חוזרת על מספר חלוקות של נתוני האימון
Data drift	התפלגות הקלט השתנתה
Data leakage	שימוש במידע שלא היה זמין ברגע החיזוי
Embedding	ייצוג מספרי נלמד של פריט, לרוב טקסט
Epoch	מעבר שלם על נתוני האימון
Feature	תכונה שמשמשת כקלט למודל
Feature Store	מערכת לניהול תכונות ולאספקתן לאימון ולחיזוי
Fine-tuning	המשך אימון של מודל קיים על נתונים ייעודיים
Hyperparameter	הגדרה של תהליך הלמידה, שאינה נלמדת ממנו
Idempotency	הרצה חוזרת אינה משנה את התוצאה
Image	תוצר שמכיל אפליקציה וסביבתה
Inference	שימוש במודל מאומן להפקת תחזית
Label	התשובה הנכונה שממנה לומדים
Latency	זמן תגובה לבקשה
Liveness probe	בדיקה אם יש להפעיל את התהליך מחדש
Model Card	מסמך שמתעד מודל: מטרה, נתונים, מגבלות ואישורים
Model Registry	מערכת לניהול זהות וגרסאות של מודלים

מונח	פירוש
Overfitting	התאמת יתר לנתוני האימון על חשבון הכללה
Pipeline	רצף שלבי עיבוד ומודל, שנשמרים ומופעלים יחד
Point-in-time correctness	חיבור תכונות לערכן בזמן ההיסטורי הנכון
Precision	איזה חלק מההתרעות היה מוצדק
Prompt	ההוראות והקלט שניתנים למודל שפה
RAG	שליפת מידע רלוונטי והוספתו להקשר של מודל שפה
Readiness probe	בדיקה אם אפשר לשלוח תעבורה למופע
Recall	איזה חלק מהמקרים האמיתיים זוהה
Regularization	הגבלת פתרון הלמידה כדי לשפר הכללה
Rollback	חזרה לגרסה קודמת שעבדה
Schema	מבנה הנתונים המוסכם: שדות, סוגים ואילוצים
Shadow deployment	גרסה חדשה מקבלת עותק בקשות בלי להשפיע על התשובה
SLI / SLO	המדד עצמו, לעומת היעד שנקבע לו
Throughput	כמות בקשות ליחידת זמן
Token	יחידת טקסט בסיסית שמודל שפה מעבד
Training-serving skew	פער בין העיבוד באימון לעיבוד בשירות
Underfitting	המודל פשוט מדי מכדי לתפוס את המבנה
Validation set	נתונים לבחירת מודל והגדרות

ב' צ'קליסט שחרור מודל לייצור

עבור על הרשימה לפני כל שחרור. הסימון נשמר בדפדפן שלך.

1 • נתונים ובעיה

- ☐ רגע החיזוי מוגדר, ומתועד בכתב
- ☐ כל תכונה נבדקה מול השאלה "האם היא הייתה זמינה אז?"
- ☐ שיטת החלוקה מתאימה לשימוש האמיתי במערכת
- ☐ סט הבדיקה נבדק פעם אחת, בסוף
- ☐ גרסת מאגר הנתונים מתועדת (hash או snapshot)

2 • מודל והערכה

- ☐ קיים Baseline, והמודל מנצח אותו במידה שמצדיקה את המורכבות
- ☐ המדד נבחר לפי ההחלטה העסקית, לא לפי נוחות
- ☐ נמדדו ביצועים גם לפי קבוצות, ולא רק ממוצע
- ☐ בוצע ניתוח טעויות על הדוגמאות הגרועות ביותר
- ☐ סף ההחלטה נבחר במפורש ומתועד

3 • הנדסה

- ☐ העיבוד והמודל נשמרים יחד כ-Pipeline
- ☐ סדר התכונות נשמר במטא-דאטה ונאכף בשירות
- ☐ חוזה הקלט דוחה שדה לא מוכר וערך מחוץ לטווח
- ☐ יש בדיקות לקוד, לחוזה ולמודל, והן רצות ב-CI
- ☐ גרסאות התלויות נעוצות
- ☐ ה-Image נבנה ונבדק בבדיקת עשן

4 • תפעול

- ☐ יש `/health` ו-`/ready` נפרדים
- ☐ כל תחזית נרשמת עם מזהה, גרסת מודל וזמן
- ☐ מוגדרים מדדי שירות ויעדים (SLI/SLO)
- ☐ מוגדר ניטור לנתונים ולהתפלגות התחזיות
- ☐ ידוע מתי ואיך מגיעות התשובות האמיתיות
- ☐ תהליך Rollback נבדק בפועל, לא רק תועד

5 • אבטחה וממשל

- ☐ לשירות יש הרשאות מינימליות בלבד
- ☐ סודות מנוהלים מחוץ לקוד ומחוץ ל-Image
- ☐ אין מידע אישי מיותר בלוגים
- ☐ מתועד מי אישר את הגרסה ומתי
- ☐ מקור המודל ידוע ומהימן

ג' מקורות למידה

רשימה קצרה ומכוונת. עדיף לסיים שלושה מקורות מאשר לפתוח שלושים.

נושא	מקור
Python מאפס	CS50P · Harvard
כלי עבודה, Git ו-Shell	Missing Semester · MIT
SQL ומסדי נתונים	PostgreSQL Tutorial
יסודות ML	Google ML Crash Course
עבודה מעשית ב-scikit-learn	User Guide · Common Pitfalls
בניית שירותים	FastAPI
קונטיינרים	Docker Get Started
ניהול ניסויים	MLflow
תזמור נתונים	Airflow
תשתית	Kubernetes Concepts
אמינות ותפעול	Google SRE Book
למידה עמוקה	PyTorch Basics
LLM ו-NLP	Hugging Face LLM Course
ענן	AWS SageMaker AI

כל אחת מהן מופיעה במדריך. כאן הן במקום אחד, כדי לחזור עליהן לפני ראיון.

#	הטעות	פרק
1	ללמוד עשרים כלים במקום להבין את הבעיות שהם פותרים	1
2	לקרוא קוד בלי להריץ אותו	00
3	להתקין חבילה בסביבה אחת ולהריץ קוד בסביבה אחרת	3
4	לדחוף סודות או קובץ <code>.env</code> ל-Git	6
5	לבצע JOIN בלי לבדוק ייחודיות מפתח, ולהכפיל רשומות	7
6	לבנות מודל בלי Baseline להשוואה	11
7	לבלבל בין מתאם לסיבתיות ולהמליץ המלצה מזיקה	11
8	להשתמש בשדה שעודכן אחרי רגע החיזוי	12
9	לחשב Scaling או השלמות לפני חלוקת הנתונים	12
10	לחלק אקראית כשהשימוש האמיתי הוא לקוחות חדשים או עתיד	13
11	להסתכל על סט הבדיקה שוב ושוב ולבחור לפיו	13
12	לדווח Accuracy על מאגר לא מאוזן	15
13	למחוק חריגים בלי לבדוק מה הם מייצגים	16
14	לנסות אלגוריתם נוסף במקום לקרוא את הטעויות	17
15	לטעון את המודל מחדש בכל בקשת API	19
16	לקבל שדה לא מוכר בשקט במקום לדחות אותו	19
17	להסתמך על <code>latest</code> כזיהוי גרסה	23
18	לבצע Rollback למודל בלבד, בלי הקוד והעיבוד	25
19	להסיק מ-drift שצריך לאמן מחדש, בלי לחקור	29
20	להשאיר משאבי ענן פתוחים כי "הם לא בשימוש"	36

סוף המדריך, תחילת העבודה

אם הגעת לכאן בקריאה – יש לך מפה. אם הגעת לכאן אחרי שהרצת את הקוד, בנית את הפרויקטים, ושברת אותם בכוונה כדי לראות מה קורה – יש לך מקצוע.

ההבדל בין השניים הוא כמה חודשים של עבודה עקבית. הוא לא ההבדל בין אנשים.

מאפס למומחה · ML Engineering ו־MLOps – מדריך מעשי, מהדורה שנייה.

קוד המעבדה נמצא ב־ `labs/iris-service` ונבדק בפועל. הדוגמאות המספריות הן לימודיות; אין להסיק מהן ביצועים של מערכת אמיתית.